



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki

Nr albumu: 189788

Poziom kształcenia: studia drugiego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Elektronika i telekomunikacja

Specjalność: Systemy wbudowane czasu rzeczywistego

PRACA DYPLMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Badanie wpływu różnych metod optymalizacji kodu na wydajność systemu wbudowanego opartego na systemie Linux.

Tytuł pracy w języku angielskim: Investigation of the impact of various code optimization methods on the performance of a Linux-based embedded system.

Opiekun pracy: dr hab. inż. Jacek Marszał

STRESZCZENIE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Słowa kluczowe: Optymalizacja algorytmów, systemy wbudowane, system operacyjny Linux

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna,

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Keywords: Self-organizing maps, unsupervised learning, clustering, classification, WTM algorithm

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering,

SPIS TREŚCI

Spis treści	5
1 Wstęp i cel pracy	8
1.1 Cel pracy	8
1.2 Zawartość rozdziałów	8
2 Architektura systemów operacyjnych w zastosowaniach wbudowanych	9
2.1 Rola systemów operacyjnych w systemach wbudowanych	9
2.1.1 Model warstwowy systemów operacyjnych	10
2.2 Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych	11
2.2.1 Tryby pracy procesora	12
2.2.2 Procesy	12
2.2.3 Wątki	12
2.2.4 Szeregowanie zadań	13
2.2.5 Pamięć wirtualna	13
2.3 Systemy wbudowane czasu rzeczywistego wraz z ich podziałem	13
2.4 Architektura struktur jądra systemów operacyjnych	15
2.5 Studium przypadku systemu operacyjnego Linux	16
2.5.1 Procesy, wątki w systemie Linux	17
2.5.2 Szeregowanie i synchronizacja w systemie Linux	17
2.5.3 Zarządzanie pamięcią	18
3 Algorytm przetwarzania danych i techniki optymalizacji programów komputerowych	19
3.1 Opis i analiza Szybkiej Transformaty Fouriera	19
3.1.1 Podstawy matematyczne algorytmu FFT	19
3.1.2 Szybka transformata Fouriera	21
3.2 Opis procesu kompilacji programu komputerowego	25
3.2.1 Procesory języka programowania	25
3.2.2 Szczegółowy opis procesu kompilacji	25
3.2.3 Wewnętrzna struktura kompilatora	27
3.3 Przegląd najpopularniejszych kompilatorów języka C	28
3.3.1 Kompilator GCC	28
3.3.2 Kompilator Clang	28
3.4 Zaawansowane techniki optymalizacji kodu i flagi kompilacji	29
3.4.1 Przekształcenia optymalizujące	30
3.4.2 Hierarchia pamięci i pamięć podręczna	31
3.4.3 Podstawowe flagi optymalizacyjne	31
4 Projekt systemu testowego i metodyka przeprowadzonych badań	34
4.1 Zaimplementowane warianty algorytmu FFT	34
4.2 Opis platformy sprzętowej	34
4.2.1 Jednostka Neon	34

4.3	Założenia projektowe i architektura aplikacji	34
4.3.1	Weryfikacja poprawności obliczeń z biblioteka zewnętrzna	34
4.4	Użyte biblioteki i narzędzia programistyczne	34
4.5	Metodyka przeprowadzonych pomiarów	34
5	Analiza wyników przeprowadzonych badań	35
5.1	Analiza wpływu flag kompilatora na kod sekwencyjny	35
5.2	Analiza skalowalności rozwiązania wielowątkowego	35
5.3	Zestawienie zbiorcze i dyskusja wyników	35
6	Podsumowanie i wnioski	36
6.1	Wnioski końcowe	36
6.2	kierunek dalszych badań	36
	Spis rysunków	37
	Spis tabel	38

LISTA SYMBOLI

LISTA SKRÓTÓW

1. WSTĘP I CEL PRACY

1.1. Cel pracy

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Charakterystyka systemów wbudowanych opartych na systemie Linux.**
Obejmuje charakterystykę systemów wbudowanych opartych na systemie Linux, z opisem podstawowych pojęć jak jądro systemu, menadżer procesów, zarządzanie pamięcią oraz mechanizmy wielozadaniowości. Opisuje specyfikę zarządzania zasobami sprzętowymi przez jądro systemu. Szczególną uwagę poświęcono mechanizmom wielozadaniowości, szeregowania wątków oraz zarządzania pamięcią wirtualną, które mają kluczowy wpływ na wydajność aplikacji.
- **Rozdział 3: Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu.**
Przedstawia teoretyczne podstawy algorytmu Szybkiej Transformaty Fouriera (FFT) oraz analizę jego złożoności obliczeniowej. Prezentuje różne implementacje tego algorytmu. Omawia proces kompilacji kodu w języku C, nakreślając istotę wyboru kompilatora. Ponadto przedstawia poziomy optymalizacji oferowane przez kompilator GCC, używany w pracy. Wyjaśnia zaawansowane techniki programistyczne, takie jak instrukcje wektorowe oraz optymalizacja pod kątem dostępu do pamięci podręcznej.
- **Rozdział 4: Projekt i implementacja aplikacji testowej.**
Opisuje szczegółowo opracowane oprogramowanie realizujące algorytm Szybkiej Transformaty Fouriera (FFT). Przedstawia alternatywne warianty implementacji kodu: wersję sekwencyjną, wersję zoptymalizowaną pod kątem dostępu do pamięci oraz wersję wielowątkową wykorzystującą mechanizmy współbieżności.
- **Rozdział 5: Środowisko badawcze i metodyka pomiarów.**
Charakteryzuje wykorzystaną platformę sprzętową oraz konfigurację systemu operacyjnego Linux. Opisuje narzędzia programistyczne, biblioteki oraz frameworki użyte do implementacji i optymalizacji kodu. Definiuje scenariusze testowe, przygotowane konfiguracje procesu kompilacji (zestawy flag optymalizacyjnych) oraz narzędzia i metody użyte do pomiaru czasu wykonania i zużycia zasobów systemowych.
- **Rozdział 6: Analiza wpływu optymalizacji na wydajność systemu.**
Prezentuje wyniki przeprowadzonych badań. Każdy podrozdział zawiera zestawienie i analizę wpływu konkretnych czynników – poziomu optymalizacji kompilatora, zmian w kodzie źródłowym oraz liczby wątków – na ostateczną wydajność aplikacji w systemie wbudowanym.
- **Rozdział 7: Podsumowanie i wnioski.**
Stanowi syntezę uzyskanych wyników oraz ocenę skuteczności badanych metod optymalizacji. Zawiera wnioski końcowe dotyczące relacji między nakładem pracy programisty a automatyczną optymalizacją kompilatora oraz propozycje dalszych kierunków badań.

2. ARCHITEKTURA SYSTEMÓW OPERACYJNYCH W ZASTOSOWANIACH WBUDOWANYCH

Projektowanie złożonych systemów komputerowych, zwłaszcza takich jak rozwiązania bazujące na implementacji algorytmów przetwarzania sygnałów w rygorze czasu rzeczywistego przy jednocześnie ograniczonych zasobach sprzętowych, jest zadaniem, które wymaga wielu przemyślanych decyzji przy doborze architektury tworzonego systemu.

Wybór odpowiedniej platformy programowej jest jedną z kluczowych decyzji projektowych. Projektant systemu wbudowanego staje wówczas często przed wyborem między podejściem bezsystemowym, zapewniającym pełną kontrolę nad sprzętem, a zastosowaniem systemu operacyjnego, który wprowadza warstwę abstrakcji kosztem częściowej utraty determinizmu czasowego oraz zwiększenia zużycia fizycznych zasobów systemu [1]. Decyzja ta ma bezpośredni wpływ na poprawność systemu, nie tylko z uwagi na osiąganą wydajność obliczeniową, ale również poprzez bezpieczeństwo i stabilność systemu. W celu zwiększenia jakości projektowanego systemu inżynier oprogramowania powinien posiadać niezbędną wiedzę teoretyczną z zakresu działania systemów operacyjnych, aby zagwarantować tworzenie możliwie wydajnych i niezawodnych systemów komputerowych.

Celem prezentowanego rozdziału jest usystematyzowanie wiedzy niezbędnej do zrozumienia, w jaki sposób architektura stosowanego systemu operacyjnego wpływa na wydajność uruchamianych na nim programów. Rozważania rozpoczynają się od przedstawienia warstwowego modelu systemów operacyjnych oraz kluczowych mechanizmów jądra, takich jak tryby pracy procesora, procesy, wątki, szeregowanie zadań, pamięć wirtualna oraz synchronizacja współbieżnych operacji. Na tej podstawie omówiony zostaje podział systemów wbudowanych czasu rzeczywistego, a także dwa podstawowe modele budowy jądra: architektura monolityczna oraz mikrojądrowa. Ostatnia część rozdziału zawiera studium przypadku systemu operacyjnego Linux, w którym w skrócony sposób starano się przedstawić niezbędne szczegóły implementacji mechanizmów systemowych.

2.1. Rola systemów operacyjnych w systemach wbudowanych

Podczas projektowania systemu wbudowanego, włączając w to system wbudowany czasu rzeczywistego, projektant staje przed wyborem jednego z dwóch podstawowych podejść do organizacji i kontroli całego środowiska. Pierwszym z nich jest **podejście bezsystemowe** (ang. *bare-metal*), w którym całe oprogramowanie funkcjonuje bez żadnej warstwy pośredniej, mając bezpośredni i wyłączny dostęp do fizycznych układów procesora oraz peryferiów. Choć pozwala to na pełną kontrolę nad sprzętem i minimalne narzuty czasowe, to drastycznie ogranicza skalowalność każda zmiana komponentu lub rozbudowa funkcji wymusza wówczas głęboką modyfikację struktury całego kodu.

Alternatywą jest zastosowanie **systemu operacyjnego**, który możemy zdefiniować jako podstawowe oprogramowanie pośredniczące między sprzętem komputerowym a użytkownikiem, którego głównym zadaniem jest zarządzanie i przydzielanie zasobów sprzętowych oraz tworzenie środowiska pozwalającego na wykonywanie programów zaimplementowanych przez jego użytkownika [2].

Jak wynika z przytoczonej definicji, stosowanie systemu operacyjnego pomaga rozwiązy-

wać niektóre problemy napotymane przy stosowaniu podejścia bezsystemowego. Dzieje się tak poprzez całościowe odizolowanie warstwy programowej od fizycznych struktur sprzętu. System operacyjny działa jako nadrzędny pośrednik, przejmując na siebie zadanie nadzorowania środowiska wykonawczego i separując logikę działania od konkretnej platformy sprzętowej.

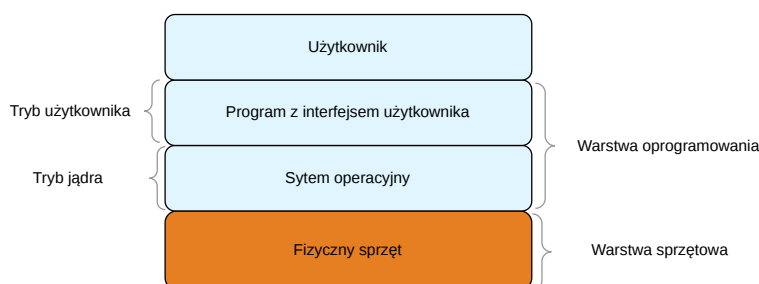
Aby w pełni zrozumieć, jak system operacyjny realizuje te zadania w środowisku wbudowanym, konieczne jest przyjrzenie się jego wewnętrznej architekturze. W dalszych sekcjach zostaną szczegółowo omówione kluczowe mechanizmy oraz podstawowa struktura ogólnych systemów operacyjnych.

2.1.1. Model warstwowy systemów operacyjnych

W inżynierii oprogramowania istotną rolę odgrywa koncepcja **abstrakcji**, polegająca na celowym ukryciu szczegółów implementacji danej funkcjonalności oprogramowania, przed jej potencjalnym użytkownikiem [1]. Podejście to opiera się na założeniu, że należy dostarczyć gotową funkcjonalność wraz z odpowiednią dokumentacją opisującą sposób jej wykorzystania, zamiast wymagać od użytkownika zrozumienia wewnętrznych mechanizmów działania wytworzonego modułu.

Taka praktyka w sposób znaczący podnosi wygodę wytwarzania nowych elementów systemu, umożliwia zrównoleglenie wytwarzania oprogramowania oraz pozwala na większą specjalizację jej producentów. W celu realizacji komunikacji między kolejnymi modułami systemu stosuje się takie narzędzia jak **interfejsy programistyczne** (ang. *Application Programming Interface*, skrót **API**), czyli zbiór ściśle określonych reguł i sposobów komunikacji, które umożliwiają komunikację między różnymi warstwami oprogramowania. Narzędzie to jest niezbędne, aby w sposób przewidywalny i bezpieczny przekazywać dane lub sterowanie, między różnymi elementami systemu.

Pojęcie abstrakcji jest kluczowe dla zrozumienia architektury współczesnych systemów operacyjnych. W literaturze spotyka się różne podejścia do klasyfikacji poziomów systemu komputerowego [1, 2], jednak ich cechą wspólną jest wyraźne odseparowanie zasobów fizycznych od uruchamianego na nich oprogramowania. Na potrzeby niniejszej pracy przyjęto strukturę opartą na propozycji przedstawionej w [2], którą w graficzny sposób zaprezentowano na rysunku 2.1.



Rysunek 2.1: Warstwowy model systemu komputerowego

Jak pokazano na schemacie, najniższy poziom stanowi warstwa sprzętowa, do której zalicza się wszystkie fizyczne komponenty urządzenia, takie jak **jednostka centralna** (ang. *Central Processing Unit*, CPU), **pamięć** oraz **urządzenia wejścia-wyjścia** (ang. *Input/Output Devices*, I/O). Ponad nią znajduje się warstwa oprogramowania, na którą składa się zarówno system operacyjny, jak i programy użytkownika. Choć oba te elementy należą do tej samej warstwy, funkcjonują one na innych zasadach. Odróżnia je praca w różnych trybach procesora, różniących

się poziomem uprawnień oraz zakresem dostępu do fizycznych urządzeń. Tryby te nazywają się odpowiednio **trybem jądra** (ang. *kernel mode*), dla systemu operacyjnego, oraz **trybem użytkownika** (ang. *user mode*), dla aplikacji użytkownika. Szczegółowe omówienie przytoczonych trybów oraz mechanizmów ich przełączania zostanie przedstawione w dalszych częściach tego rozdziału.

Warto przy tym podkreślić, że w przytoczonej pozycji literatury [2] jako integralny komponent struktury systemu komputerowego uwzględnia się jego użytkownika. Zabieg ten ma na celu jednoznaczne wskazanie, że przy projektowaniu dowolnego systemu komputerowego należy zawsze pamiętać o jego ostatecznym przeznaczeniu oraz o użytkowniku końcowym.

W modelu warstwowym architektury komputerowej system operacyjny pełni szczególną rolę, zarządzając granicą między aplikacjami a warstwą fizyczną. W celu zapewnienia poprawnego i przewidywalnego działania systemu wprowadza poziomy abstrakcji, które są niezbędne z kilku istotnych powodów:

1. **Konieczność zapewnienia bezpieczeństwa** - Współczesne systemy operacyjne charakteryzują się ogromną złożonością, a ich kod źródłowy sięga kilkudziesięciu milionów linii [1]. Izolacja poszczególnych poziomów wprowadza wyraźną hierarchię uprawnień. Dzięki temu błąd powstały w kodzie aplikacyjnym pozostaje odizolowany i nie wpływa bezpośrednio na stabilność pozostałych funkcjonalności systemowych ani na pracę samego sprzętu.
2. **Zapewnienie przenoszalności oprogramowania** - Zastosowanie warstw pośrednich sprawia, że oprogramowanie nie jest bezpośrednio uzależnione od konkretnych urządzeń fizycznych. Pozwala to na uruchamianie tych samych programów na różnorodnych platformach sprzętowych, umożliwiając przy tym niezależny rozwój warstwy aplikacyjnej i sprzętowej.
3. **Umożliwienie korzystania z narzędzi wysokiego poziomu** - Ukrycie technicznych szczegółów pracy ze sterownikami lub rejestrami pozwala na tworzenie kodu w językach wysokiego poziomu. Programista może skupić się na logice i strukturze zgodnej ze sposobem myślenia człowieka, zamiast na bezpośredniej, niskopoziomowej obsłudze układów elektronicznych.

2.2. Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych

Pomimo dużej różnorodności współczesnych systemów operacyjnych, ich architektura opiera się na spójnym zestawie uniwersalnych abstrakcji i mechanizmów. Od strony koncepcyjnej rozwiązania te wprowadzają ład w złożonym środowisku sprzętowym, przybliżając sposób funkcjonowania komputera do logiki myślenia i pracy człowieka.

Koncepcje opisane w prezentowanej sekcji pozwalają na stworzenie przejrzystego i uporządkowanego modelu zarządzania zasobami w systemie operacyjnym. Zrozumienie tych bazowych koncepcji jest kluczowe dla analizy wydajności, ponieważ to one determinują sposób, w jaki program aplikacji, stworzonej przez użytkownika, jest uruchamiany i wykonywany na fizycznym sprzęcie.

Na potrzeby niniejszej pracy konieczne jest przedstawienie następujących pojęć i mechanizmów:

- Tryby pracy procesora
- Proces
- Wątek

- Szeregowanie zadań
- Pamięć wirtualna

2.2.1. Tryby pracy procesora

Współczesne komputery realizują ochronę zasobów poprzez tryby pracy procesora [2], z których kluczowe to **tryb użytkownika** (ang. user mode) oraz **tryb jądra** (ang. kernel mode). Aplikacje użytkownika funkcjonują w trybie użytkownika, nieuprzywilejowanym, co uniemożliwia im między innymi, bezpośredni dostęp do urządzeń peryferyjnych oraz fizycznej pamięci.

Wszelkie operacje wymagające kontroli nad sprzętem realizowane są za pośrednictwem **wywołań systemowych** [2] (ang. system calls), które przełączają procesor w uprzywilejowany tryb jądra.

2.2.2. Procesy

Proces (ang. process) można rozumieć jako wykonujący się program komputerowy [2]. Jego kluczową cechą jest przydzielona, odizolowana od innych procesów wirtualna **przestrzeń adresowa**. Jej wprowadzenie ma zagwarantować bezpieczeństwo – błąd w obrębie jednego procesu nie wpływa bezpośrednio na stabilność procesów systemowych i innych procesów użytkownika.

Współczesne systemy komputerowe składają się z licznych komponentów i aplikacji, co sprawia, że na wspólnych, fizycznych zasobach sprzętowych funkcjonuje jednocześnie wiele procesów. Ze względu na to, że pojedynczy rdzeń procesora może w danej chwili realizować tylko jedno zadanie, system operacyjny stosuje przełączanie kontekstu, cyklicznie przydzielając czas procesora poszczególnym procesom. Ponieważ każdy proces operuje w odrębnej przestrzeni adresowej, zmiana ta wymaga przeładowania pamięci operacyjnej. W efekcie przełączanie kontekstu między procesami jest operacją stosunkowo czasochłonną i wiąże się z zauważalnym opóźnieniem, które można nazywać też **narzutem systemu operacyjnego** [1].

2.2.3. Wątki

Wątek (ang. thread) stanowi wyodrębnioną jednostkę wykonawczą (ciąg instrukcji) uruchamianą na procesorze, funkcjonującą w obrębie konkretnego procesu [1].

W ramach jednego procesu może działać wiele wątków wykonujących się we wspólnej przestrzeni adresowej. Wątki należące do tego samego procesu współdzielą ze sobą zasoby systemowe (m.in. przestrzeń adresową, zmienne globalne czy otwarte pliki), jednak każdy z nich posiada własny, prywatny kontekst wykonania [1], na który składają się:

- **Licznik instrukcji** (ang. Program Counter),
- **Zestaw rejestrów procesora**,
- **Prywatny stos** (przechowujący zmienne lokalne oraz historię wywołań funkcji),
- **Stan wątku** (np. uruchomiony, zablokowany, gotowy).

Dzięki współdzieleniu przestrzeni adresowej wątki generują znacznie mniejszy narzut czasowy niż procesy, przy przełączaniu, wymagając jednak stosowania odpowiednich mechanizmów synchronizacji.

Zarządzanie wątkami może być realizowane w przestrzeni użytkownika lub bezpośrednio w jądrze systemu. W celu zapewnienia przenoszalności oprogramowania wielowątkowego między różnymi środowiskami operacyjnymi stworzony został standard **POSIX Threads** (znany jako **Pthreads**, zdefiniowany w normie IEEE 1003.1c) [1]. Definiuje on uniwersalne interfejsy programistyczne przeznaczone m.in. do tworzenia, kończenia oraz synchronizacji pracy wątków.

2.2.4. Szeregowanie zadań

Szeregowanie zadań odpowiada za zarządzanie przydziałem czasu procesora. Ponieważ pojedynczy rdzeń CPU nie może wykonywać wielu zadań jednocześnie, **planista** (ang. *scheduler*) decyduje o kolejności i czasie uruchamiania wątków i procesów na podstawie określonych algorytmów [2]. Mechanizm ten zapewnia sprawiedliwy dostęp do zasobów i płynność działania systemu, jednak częsta zmiana wykonywanych zadań generuje bezpośredni narzut czasowy związany z przełączaniem kontekstu.

2.2.5. Pamięć wirtualna

Pamięć wirtualna sprawia, że procesy nie operują bezpośrednio na fizycznej pamięci operacyjnej, lecz na wirtualnej przestrzeni adresowej przydzielanej i mapowanej na sprzęt przez **jednostkę zarządzania pamięcią (MMU, ang. Memory Management Unit)**[1].

W kontekście pamięci wirtualnej kluczowym mechanizmem jest **stronicowanie** [1]. Polega ono na podziale wirtualnej przestrzeni adresowej programu na bloki o stałym rozmiarze, zwane stronami (ang. *pages*), które są dynamicznie odwzorowywane na fizyczną pamięć operacyjną. Operacje tę wykonuje jednostka MMU. Mechanizm zarządzania pamięcią wirtualną został opisany, na przykładzie systemu Linux w sekcji 2.5.3

2.3. Systemy wbudowane czasu rzeczywistego wraz z ich podziałem

Systemem wbudowanym nazywamy dedykowany system komputerowy, który stanowi integralną część większego urządzenia lub maszyny i jest zaprojektowany do wykonywania ściśle określonego zadania [1]. W przeciwieństwie do komputerów ogólnego przeznaczenia (takich jak komputery osobiste czy smartfony), na których użytkownik może uruchamiać dowolne aplikacje, system wbudowany ma z góry zdefiniowaną funkcję i nie jest przeznaczony do późniejszych, swobodnych modyfikacji przez użytkownika końcowego.

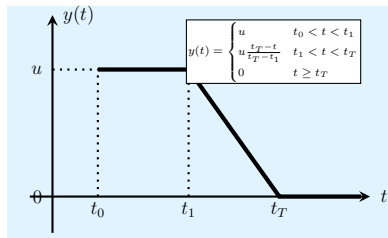
Szczególną grupą w obrębie systemów wbudowanych są **systemy wbudowane czasu rzeczywistego**. Są to takie systemy, których poprawność wyniku przetwarzania nie zależy tylko od poprawności logicznej odpowiedzi, ale też od czasu przetwarzania, poprzedzającego odpowiedź systemu [3]. W przypadku niewypracowania odpowiedzi w zadanym czasie zdarzenie to jest traktowane jako błąd systemu, co w skrajnych przypadkach może prowadzić do awarii lub uszkodzenia fizycznego sprzętu.

Systemy czasu rzeczywistego są powszechnie stosowane w zastosowaniach krytycznych, takich jak przemysł zbrojeniowy, aparatura medyczna czy branża motoryzacyjna, co wymaga wprowadzenia ścisłych kryteriów oceny ich niezawodności. W celu jednoznacznego zdefiniowania wymagań dotyczących determinizmu czasowego, w literaturze wprowadza się klasyfikacje systemów ze względu na rygor zachowania narzuconych terminów. W literaturze [1] systemy czasu rzeczywistego dzieli się na:

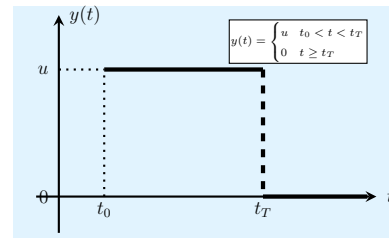
- **Systemy twarde (Hard real-time systems)**

- Systemy miękkie (Soft real-time systems)
- Systemy stabilne (Firm real-time systems)

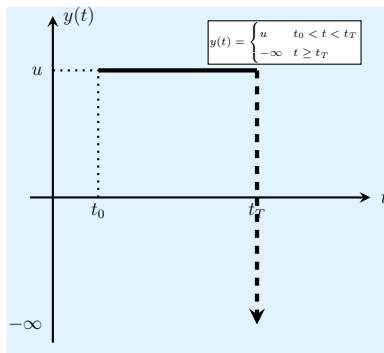
W celu lepszego zobrazowania podziału systemów czasu rzeczywistego, na rysunku rys. 2.2 przedstawiono wykresy funkcji przydatności odpowiedzi $y(t)$ od czasu jej wypracowania przez system [1].



(a) Miękki system czasu rzeczywistego



(b) Solidny system czasu rzeczywistego



(c) Twardy system czasu rzeczywistego

Rysunek 2.2: Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].

Przed przystąpieniem do analizy poszczególnych wykresów, konieczne jest zdefiniowanie parametrów oraz skrótów stanowiących zmienne opisujące te funkcje:

- t – oś czasu, reprezentująca moment dostarczenia odpowiedzi przez system,
- $y(t)$ – funkcja przydatności (wartości) odpowiedzi dla obsługiwanego procesu,
- u – maksymalna, nominalna przydatność prawidłowo wypracowanego wyniku,
- t_0 – moment pojawienia się zdarzenia lub żądania wypracowania odpowiedzi przez system,
- t_1 – czas graniczny, do którego odpowiedź zachowuje pełną wartość (w systemach miękkich),
- t_T – ostateczny limit czasowy (ang. *deadline*), po przekroczeniu którego odpowiedź nie ma już pełnej wartości.

Analizując przebiegi funkcji przydatności dla poszczególnych klas systemów, można precyzyjnie scharakteryzować ich zachowanie.

Najbardziej restrykcyjne kryteria czasowe są charakterystyczne dla twardych systemów czasu rzeczywistego, co zobrazowano na rysunku 2.2c. Ich stosowanie gwarantuje bezwzględnie

terminowe wypełnienie zadań krytycznych przed upływem limitu t_T . Przekroczenie czasu przetwarzania powoduje natychmiastowy spadek funkcji przydatności do wartości $-\infty$. W przypadku rozwiązań spóźniona odpowiedź jest nie tylko całkowicie bezużyteczna, ale jej pojawienie się po wyznaczonym terminie jest najczęściej równoznaczne z wystąpieniem krytycznego błędu, który może doprowadzić do poważnej awarii lub fizycznego uszkodzenia całego systemu.

Kolejną z omawianych grup stanowią miękkie systemy czasu rzeczywistego, których zachowanie zilustrowano na rysunku 2.2a. Systemy te cechują się mniejszym rygorem w kwestii terminowości, co oznacza, że zadania są wykonywane tak szybko, jak to możliwe, a ewentualne opóźnienia nie niosą za sobą skutków katastrofalnych. Jeśli odpowiedź zostanie dostarczona po czasie t_1 , ale przed ostatecznym limitem t_T , jej przydatność maleje, powodując jedynie spadek jej jakości. Przy przetwarzaniu przekraczającym czas t_T przydatność odpowiedzi wynosi zero.

Solidne systemy czasu rzeczywistego stanowią pewnego rodzaju kompromis pomiędzy twardymi a miękkimi systemami czasu rzeczywistego. Zachowanie funkcji przydatności systemów solidnych przedstawiono na rysunku 2.2b. Każda odpowiedź dostarczona w przedziale od t_0 do t_T zachowuje stałą, pełną wartość u . W momencie przekroczenia terminu granicznego t_T przydatność wyniku natychmiastowo i skokowo spada do zera. Choć fakt spóźnienia powoduje całkowitą nieprzydatność rozwiązania, to sytuacja ta wciąż nie generuje bezpośredniego zagrożenia dla stabilności czy integralności samego systemu.

Zdefiniowane powyżej rygory czasowe określają teoretyczne oczekiwania wobec systemu czasu rzeczywistego. W praktyce realizacja tych założeń sprowadza się do oceny zasadności stosowania **systemu operacyjnego czasu rzeczywistego** w zderzeniu z bezpośrednim programowaniem sprzętu. Decyzja ta bezpośrednio determinuje strukturę i elastyczność budowanego środowiska.

2.4. Architektura struktur jądra systemów operacyjnych

Systemy operacyjne można klasyfikować według wielu kryteriów, z których jednym z najczęściej spotykanym jest podział, ze względu na ich docelowe przeznaczenie. Przy takim podziale możemy wyróżnić systemy operacyjne generalnego przeznaczenia, systemy operacyjne rozproszone, systemy operacyjne komputerów mainframe, systemy operacyjne serwerów oraz wiele innych [1].

Jednakże, w kontekście analizy wydajności kluczowy jest podział ze względu na strukturę systemu, która określa architekturę jego jądra. Na tym etapie jądro systemu możemy zdefiniować jako jedyny program, który działa w pamięci komputera nieprzerwanie przez cały czas od jego uruchomienia, pełniąc funkcję pomostu komunikacyjnego między uruchomionymi aplikacjami, a fizycznym sprzętem [2].

Konstrukcja jądra decyduje o sposobie komunikacji między usługami systemowymi, a warstwą aplikacyjną, co bezpośrednio wpływa na narzuty czasowe oraz efektywność optymalizacji kodu programów. Z uwagi na to, że przedmiotem badań w niniejszej pracy jest system Linux, analiza skupi się głównie na architekturze monolitycznej. Dla celów porównawczych zostanie ona zestawiona z inną szeroko spotykaną architekturą w systemach wbudowanych - strukturą mikrojądra.

W **architekturze monolitycznej** (ang. *monolithic kernel*) wszystkie główne zadania systemu operacyjnego są połączone w jeden rozbudowany program komputerowy [1]. Zarządzanie pamięcią, programami oraz wszystkimi sterownikami urządzeń odbywa się we wspólnej przestrzeni. Takie podejście sprawia, że jądro zawiera w sobie ogromną liczbę elementów, co mocno komplikuje proces jego tworzenia oraz późniejszego szukania błędów. Największą zaletą archi-

tektury monolitycznej jest jednak wysoka prędkość przetwarzania informacji – z uwagi na to, że jądro jest w całości kompilowane do jednego programu, poszczególne funkcjonalności mogą z siebie nawzajem korzystać, co znacząco przyspiesza działanie systemu. Przy architekturze monolitycznej aplikacje użytkownika są uruchamiane jako osobne procesy w chronionej pamięci. Dzięki temu jądro nie jest narażone na błędy w programach użytkownika, a różne aplikacje nie mogą nawzajem bezpośrednio mieć dostępu zasobów im nie przydzielonych. Główną wadą systemów monolitycznych pozostaje jednak słaba odporność na awarie samego jądra; jeśli błąd pojawi się w jego obrębie, na przykład w jednym ze sterowników, może dojść do awarii całego systemu.

Architektura mikrojądra (ang. *microkernel*) opiera się na zasadzie minimalizacji kodu wykonywanego z najwyższymi uprawnieniami procesora [2]. Kompletny system operacyjny składa się w tym przypadku z uproszczonego jądra oraz zestawu współpracujących ze sobą procesów systemowych. Samo mikrojądro realizuje jedynie fundamentalne zadania, takie jak zarządzanie wątkami oraz obsługa komunikacji międzyprocesowej [1]. Wszystkie pozostałe usługi, wliczając w to sterowniki urządzeń i systemy plików, są odseparowane od jądra i funkcjonują jako niezależne programy w przestrzeni użytkownika.

Główną zaletą takiego podejścia jest wysoka stabilność środowiska – awaria oprogramowania lub sterownika nie narusza integralności samego jądra. System potrafi zrestartować uszkodzony proces bez zaburzania pracy całego systemu. Wadą tego rozwiązania jest jednak niższa wydajność, wynikająca z narzutu czasowego na ciągłą konieczność komunikacji międzyprocesowej oraz częste przełączanie kontekstu procesora.

2.5. Studium przypadku systemu operacyjnego Linux

System operacyjny Linux wywodzi się z tradycji architektonicznej rodziny systemów "Unix", czerpiąc z niej kluczowe założenia projektowe, takie jak wielozadaniowość, wielodostępność oraz podział przestrzeni pamięci [1]. Jego rozwój rozpoczął się w 1991 roku z inicjatywy Linusa Torvaldsa, a dziś stanowi on jeden z najważniejszych i najczęściej wykorzystywanych projektów tworzonych w modelu otwartego oprogramowania (ang. *open source*) [1]. Dzięki wsparciu globalnej społeczności Linux stał się najczęściej wykorzystywanym systemem komputerowym w wielu zastosowaniach, począwszy od infrastruktury serwerowej przez systemy wbudowane, aż po chmury obliczeniowe.

Z perspektywy użytkownika klasycznym i wysoce elastycznym sposobem komunikacji z systemem Linux jest powłoka (ang. *shell*), w systemach Linux reprezentowana najczęściej przez program *bash* [1]. Z technicznego punktu widzenia jest to zwykły program działający w przestrzeni użytkownika, który pełni rolę interpretera poleceń [1]. Powłoka pozwala na uruchamianie innych programów wraz z odpowiednimi parametrami. Do jej kluczowych funkcjonalności należy elastyczne przekierowywanie standardowych strumieni wejścia i wyjścia oraz łączenie pojedynczych programów w potoki (ang. *pipelines*), co pozwala na przekazywanie wyników między różnymi procesami [1]. Ponadto powłoka umożliwia asynchroniczne uruchamianie zadań w tle oraz tworzenie skryptów, które dzięki obsłudze zmiennych i instrukcji sterujących (np. pętli czy instrukcji warunkowych) pozwalają na automatyzację złożonych operacji systemowych [2].

Kluczowym elementem systemu jest jądro o strukturze monolitycznej [4], które odpowiada za bezpośrednią kontrolę nad najważniejszymi zasobami systemu. Z uwagi na tę architekturę oraz konieczność zapewnienia bezpieczeństwa, aplikacje działające w przestrzeni użytkownika nie mają bezpośredniego dostępu do krytycznych zasobów systemu. Wszelka interakcja z zasobami zarządzanymi przez system musi odbywać się za pośrednictwem zdefiniowanego interfejsu

wywołań systemowych, który w systemie Linux jest zgodny ze standardem **POSIX** [1]. Każde takie wywołanie wymusza sprzętowe przełączenie procesora do trybu jądra. Operacja ta wymaga zapisania stanu rejestrów oraz zmiany kontekstu wykonania, co wiąże się z dodatkowym narzutem czasowym.

2.5.1. Procesy, wątki w systemie Linux

W przeciwieństwie do wielu innych systemów operacyjnych, Linux nie wprowadza ścisłego, wewnętrznego rozróżnienia między procesami, a wątkami [1]. Każdy kontekst wykonania, niezależnie od tego, czy jest to jednowątkowy proces, czy jeden z wielu wątków wewnątrz procesu, jest reprezentowany w jądrze jako zadanie (ang. *task*) za pomocą struktury danych `task_struct` [4].

Klasyczne tworzenie nowego procesu odbywa się poprzez wywołanie systemowe `fork`. Aby uniknąć kosztownego i nieefektywnego kopiowania całej pamięci, Linux stosuje optymalizację kopiowania przy zapisie (ang. *copy on write*) [4]. Początkowo proces potomny otrzymuje własne tablice stron, jednak wskazują one na fizyczne strony pamięci procesu macierzystego z atrybutem tylko do odczytu. Nowa kopia strony jest alokowana w pamięci RAM dopiero w momencie, gdy któryś z procesów spróbuje dokonać na niej operacji zapisu [1].

Z kolei fundamentalną różnicą w architekturze Linuksa, w porównaniu do klasycznych systemów UNIX, jest obsługa wątków. Wprowadzono w nim wywołanie systemowe `clone`, które zaciera tradycyjną różnicę między procesami a wątkami [1], umożliwiając tworzenie nowego wątku z bardzo podobnymi cechami do klasycznego procesu.

Pozwala ono zminimalizować narzut systemowy poprzez precyzyjne sterowanie tym, co jest kopiowane, a co współdzielone. Wywołanie to przyjmuje jako argument mapę bitową, w której poszczególne flagi (np. `CLONE_FILES`, `CLONE_SIGHAND`) decydują o współdzieleniu deskryptorów plików czy tablicy obsługi sygnałów [1].

2.5.2. Szeregowanie i synchronizacja w systemie Linux

System operacyjny Linux posiada przemyślany i wysoce elastyczny mechanizm szeregowania zadań oraz synchronizacji, który obsługuje zarówno standardowe programy użytkowe, jak i zadania czasu rzeczywistego [1].

W kontekście planowania czasu procesora, jądro wyróżnia trzy główne klasy wątków: zadania czasu rzeczywistego typu FIFO, zadania czasu rzeczywistego z rotacją (ang. *round-robin*) oraz zadania standardowe [1]. Wątki czasu rzeczywistego otrzymują zawsze najwyższe priorytety (w skali od 0 do 99). Mimo swojej nazwy nie dają one ścisłych gwarancji dotrzymania terminów (ang. *deadlines*), a jedynie zapewniają bezwzględne pierwszeństwo wykonania przed zwykłymi aplikacjami, [1]. Jest to bardzo istotna cecha systemu Linux, która pokazuje, że nie jest on odpowiedni (w klasycznej, nieprzerobionej wersji systemu) do bezpośredniego wykorzystania w twardej systemach czasu rzeczywistego [4]. Wątki typu FIFO działają aż do momentu dobrowolnego oddania procesora lub wyłączenia przez zadanie o wyższym priorytecie, podczas gdy wątkom szeregowanym cyklicznie przydzielane są dodatkowo określone kwanty czasu [1].

Z kolei standardowe zadania o niższych priorytetach (od 100 do 139) są z reguły obsługiwane przez algorytm *Completely Fair Scheduler* (CFS) [1], który optymalizuje i sprawiedliwie rozdziela czas procesora z wykorzystaniem struktury drzewa czerwono-czarnego [2].

Uzupełnieniem efektywnego zarządzania zadaniami są zróżnicowane mechanizmy synchronizacji, które zastąpiły historyczną, nieefektywną globalną blokadę jądra. Aby zminimalizować ryzyko konfliktów o dostęp do zasobów, Linux udostępnia pełen przekrój rozwiązań: od operacji

atomowych i barier pamięci, poprzez blokady wirujące (ang. *spinlocks*) [4] – optymalne, gdy wątek nie powinien być usypiany – aż po wysokopoziomowe muteksy i semaforey dla zadań mogących bezpiecznie przejść w stan oczekiwania [1].

2.5.3. Zarządzanie pamięcią

W systemie Linux pamięć jest stronicowana. W związku z tym, podstawową jednostką do zarządzania pamięcią fizyczną jest struktura o nazwie *page* [4]. Z uwagi na ograniczenia sprzętowe niektórych architektur, pamięć fizyczna została logicznie podzielona na strefy (ang. *zones*). Należą do nich między innymi *ZONE_DMA*, przeznaczona na operacje bezpośredniego dostępu do pamięci, *ZONE_NORMAL*, zawierająca standardowe strony, oraz *ZONE_HIGHMEM* [1].

Każdy proces uruchomiony w systemie operuje na własnej wirtualnej przestrzeni adresowej, która logicznie dzieli się na trzy główne segmenty: kod, dane oraz stos [1]. Segment kodu przechowuje instrukcje programu i domyślnie posiada atrybut umożliwiający jedynie jego odczyt, co pozwala na bezpieczne współdzielenie go między wieloma jednoczesnymi uruchomieniami tego samego programu [2]. Segment danych przechowuje zmienne globalne i statyczne, przy czym dla zmiennych niezainicjowanych Linux stosuje optymalizację: mapuje je na jedną fizyczną, wyzerowaną stronę, dopóki proces nie spróbuje dokonać modyfikacji, co wyzwala mechanizm kopiowania przy zapisie (ang. *copy-on-write*) [1]. Ponadto jądro oferuje mechanizm mapowania plików w pamięci, co umożliwia procesom bezpośredni dostęp do danych plikowych lub współdzielonych bibliotek z pominięciem standardowych wywołań systemowych [1].

Do zarządzania przydziałem fizycznych stron pamięci Linux wykorzystuje **algorytm bliźniaków** (ang. *buddy algorithm*) [1]. Choć mechanizm ten jest w większości przypadków wysoce wydajny, może prowadzić do zjawiska fragmentacji wewnętrznej w przypadku konieczności alokacji struktur o rozmiarach znacznie mniejszych niż domyślne wielkości strony. Aby zminimalizować ten problem i zredukować narzut systemowy, system wdraża mechanizm dyspozytora płytowego (ang. *slab allocator*) [4], który dysponuje pamięcią poprzez algorytm bliźniaków, aby później podzielić stronę na mniejsze fragmenty i umożliwić jej zarządzanie [1].

Na poziomie interfejsu programistycznego jądra alokacja pamięci odbywa się za pomocą kilku wyspecjalizowanych funkcji do zarządzania pamięcią. Warto tutaj wspomnieć o tych najpopularniejszych, czyli *kmalloc*, gwarantująca przydział obszaru ciągłego zarówno w przestrzeni wirtualnej, jak i fizycznej [4] oraz *vmalloc* wykorzystywana do alokacji większych struktur, niewymagających ciągłości fizycznej. Ta druga jednak wiąże się z większym narzutem wydajnościowym wynikającym z konieczności modyfikacji tablic stron [4].

3. ALGORYTM PRZETWARZANIA DANYCH I TECHNIKI OPTIMALIZACJI PROGRAMÓW KOMPUTEROWYCH

Przy konstrukcji systemu przetwarzania danych szczególne znaczenie ma dobór odpowiedniego algorytmu przetwarzania oraz sposobu jego implementacji. Jest to zasadniczo główne zadanie na początku pracy, ponieważ to właśnie algorytm przetwarzania najczęściej pozwala rozwiązać zadany systemowi problem. Dopiero po podjęciu tej decyzji architekt systemu może koncentrować się na wyborze platformy programowej czy sprzętowej, które będą miały wpływ na wydajność systemu wbudowanego. Należy jednak pamiętać, że ten sam algorytm, zapisany z wykorzystaniem tego samego języka programowania, może działać z różną szybkością w zależności od tego, jak został przetłumaczony na kod maszynowy konkretnego procesora.

Na potrzeby niniejszej pracy jako przedmiot badań wybrano **szybką transformatę Fouriera (skrót: FFT)**. Algorytm ten stanowi podstawowe narzędzie cyfrowej analizy sygnałów, a jego znaczenie dla rozwoju techniki obliczeniowej trudno przecenić — w zestawieniu opublikowanym w pozycji [5] umieszczono go wśród dziesięciu algorytmów o największym wpływie na naukę i inżynierię w XX wieku. O jego wyborze zadecydowała również charakterystyka obliczeniowa: FFT wykonuje wielokrotnie te same, proste operacje arytmetyczne na dużych zbiorach danych, przez co jest wdzięcznym obiektem badań nad wpływem optymalizacji na czas wykonania programu.

Każdy program komputerowy, również implementacja FFT, składa się nie tylko z idei algorytmu, tak zrozumiałej człowiekowi, ale też z niskopoziomowych operacji na sprzęcie, które są bezpośrednio zapisane w kodzie maszynowym, powstającym w wyniku przekształcenia programu użytkownika. Translację tę wykonują translatory języka programowania, a w przypadku niniejszej pracy, ze względu na użycie **języka programowania C** – kompilator. To, jakie instrukcje maszynowe znajdują się w programie wynikowym, zależy nie tylko od zapisu w kodzie źródłowym, ale również od zastosowanych narzędzi oraz przekazanych im poleceń. Zrozumienie tych mechanizmów jest niezbędne, aby świadomie wpływać na wydajność tworzonego oprogramowania.

Celem niniejszego rozdziału jest przedstawienie wiedzy z dwóch obszarów. Pierwsza część zawiera opis matematycznych podstaw dyskretnej transformaty Fouriera oraz analizę algorytmu FFT, wraz z omówieniem jego kolejnych etapów i wariantów. Druga część poświęcona jest procesowi kompilacji: przedstawiono w niej narzędzia biorące udział w tworzeniu programu wykonywalnego, wewnętrzną budowę kompilatora oraz przegląd rozwiązań najczęściej stosowanych w systemach opartych na jądrze Linux. Rozdział zamyka omówienie technik optymalizacji kodu wraz z flagami kompilacji, które pozwalają programiście na używanie tych technik.

3.1. Opis i analiza Szybkiej Transformaty Fouriera

3.1.1. Podstawy matematyczne algorytmu FFT

Przekształcenie Fouriera (ang. *Fourier Transform*) dla sygnałów ciągłych służy do wyznaczania widma częstotliwościowego $X(j\omega)$ sygnału czasu ciągłego $x(t)$ [6]. Wzory przekształcenia prostego i odwrotnego definiuje się następująco [6]:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt, \quad x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega \quad (3.1)$$

gdzie $\omega = 2\pi f$ oznacza pulsację, a f częstotliwość wyrażoną w hercach.

Aby całka określająca $X(j\omega)$ była zbieżna, sygnał $x(t)$ musi spełniać tzw. **warunki Dirichleta**: być bezwzględnie całkowny, mieć skończoną liczbę ekstremów oraz skończoną liczbę punktów nieciągłości w dowolnym skończonym przedziale [6].

Matematyczne podstawy ciągłej transformaty Fouriera są jednak nie do końca przydatne w świecie cyfrowego przetwarzania sygnałów, właśnie z uwagi na fakt, że sygnały, na których pracują maszyny cyfrowe, są dyskretne, a nie ciągłe. W tym celu definiuje się dyskretną transformatę Fouriera, która powstaje w wyniku dyskretyzacji szeregu Fouriera dla sygnału okresowego o okresie N [6].

W systemach cyfrowych przetwarzany sygnał dostępny jest w postaci skończonego ciągu próbek $x(n)$, $n = 0, 1, \dots, N - 1$, pobranych z częstotliwością próbkowania f_{pr} . Powyższy wzór na ciągłą transformatę Fouriera nie da się więc bezpośrednio zastosować. Zamiast niego widmo sygnału wyznacza się za pomocą **dyskretnej transformacji Fouriera** (DFT, ang. *Discrete Fourier Transform*). Matematyczne przekształcenia dla prostej i odwrotnej DFT definiuje się następująco [6]:

$$X(k) = \frac{1}{N} \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N - 1 \quad (3.2)$$

$$x(n) = \sum_{k=0}^{N-1} X(k) e^{j \frac{2\pi}{N} kn}, \quad n = 0, 1, \dots, N - 1 \quad (3.3)$$

gdzie k oznacza numer składowej widma, dla której wyliczana jest aktualnie wartość transformaty. Kolejnym składowym odpowiada pulsacja $\Omega_k = 2\pi k/N$.

Wygodnym uproszczeniem powyższych zapisów jest wprowadzenie tzw. **czynnika obrotu** (ang. *twiddle factor*) [7]:

$$W_N = e^{-j \frac{2\pi}{N}} \quad (3.4)$$

co pozwala przedstawić parę wzorów DFT w zwartej postaci [6]:

$$X(k) = \frac{1}{N} \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \dots, N - 1 \quad (3.5)$$

$$x(n) = \sum_{k=0}^{N-1} X(k) W_N^{-kn}, \quad n = 0, 1, \dots, N - 1 \quad (3.6)$$

W inżynierii oprogramowania do porównywania wydajności algorytmów powszechnie wykorzystuje się **notację dużego O** (ang. *Big O notation*), opisującą tempo wzrostu liczby wykonywanych operacji wraz ze długością wektora przetwarzanych danych wejściowych N . Zapis $O(g(N))$ oznacza, że liczba wykonywanych operacji rośnie nie szybciej niż funkcja $g(N)$ pomnożona przez pewną stałą, dla odpowiednio dużych N [7].

Innymi słowy, notacja ta pomija czynniki stałe i wyrazy niższego rzędu, skupiając się na tym, jak rośnie liczba operacji wraz ze wzrostem rozmiaru danych wejściowych [7].

Dla przykładu, bezpośrednie obliczenie wzoru (3.2) dla każdego z N prążków widma wymaga N mnożeń zespolonych, co dla całego wektora $X(k)$ wymaga wykonania zasadniczo N^2 mnożeń. W takim wypadku złożoność algorytmu DFT można określić jako $O(N^2)$. Przy ciągłym przetwarzaniu długich sygnałów złożoność ta stanowi istotne ograniczenie praktyczne. Jej redukcja jest możliwa dzięki zastosowaniu **algorytmu szybkiej transformacji Fouriera** (FFT, ang. *Fast Fourier Transform*), który pozwala znacznie obniżyć koszt obliczeniowy.

3.1.2. Szybka transformata Fouriera

Redukcja złożoności obliczeniowej DFT jest możliwa dzięki zastosowaniu algorytmów określanych mianem szybkich transformacji Fouriera (FFT). Można w tym wypadku mówić o pewnym zbiorze rozwiązań, bazujących na tym samym koncepcie. Po raz pierwszy algorytm ten zaproponowano w latach sześćdziesiątych XX wieku w pracy [8]. Rodzina algorytmów FFT pozwala na znaczące obniżenie złożoności obliczeniowej, przy zachowaniu takiej samej poprawności obliczeń jak w przypadku klasycznego algorytmu DFT. FFT jest więc wydajnym sposobem na obliczenie DFT — jego złożoność obliczeniowa spada do rzędu $O(N \log_2 N)$ [6].

W pracy Cooley-Tukeya [8] zaproponowano nowe podejście do obliczania dyskretnej transformaty Fouriera, polegające na rozłożeniu długiego sygnału, o N próbkach, na kilka mniejszych grup. Następnie policzono transformatę osobno dla każdej z nich, by w ostatnim kroku połączyć wyniki w widmo całego sygnału.

Autorzy zauważyli również, że wybór N będącego potęgą liczby 2 lub 4 jest szczególnie wygodny dla komputerów działających na liczbach binarnych. Co więcej, cały algorytm można było wykonać na tej samej tablicy danych, bez potrzeby rezerwowania dodatkowej pamięci na wyniki pośrednie [8].

Zaproponowana metoda stała się inspiracją dla tworzenia kolejnych wariantów szybkiej transformaty Fouriera [6]:

- **Radix-2**,
- **radix-4**,
- **split-radix**,
- **algorytm Gooda-Thomasa**,
- **transformacja świergotowa (ang. chirp-Z)**.

Wszystkie realizują to samo przekształcenie, czyli dyskretną transformatę Fouriera, różnią się natomiast wewnętrznymi mechanizmami, takimi jak podejście do podziału danych czy długości pojedynczego bloku przetwarzanego sygnału [6]. Szczegółowy opis wariantów wykorzystanych w niniejszej pracy, wraz z ich implementacjami, przedstawiono w Sekcja 4.1.

Do dalszej analizy teoretycznej posłuży popularny wariant FFT oparty na **podstawie 2** (ang. *radix-2*), czyli podziale na dwa podzbiory, jako najprostszy do intuicyjnego wyjaśnienia. Na algorytm w tej postaci składa się kilka podstawowych zagadnień, omówionych kolejno w dalszej części podrozdziału: sposób podziału danych, uporządkowanie próbek przed rozpoczęciem obliczeń oraz elementarna operacja arytmetyczna, powtarzana na każdym etapie.

Podział wektora wejściowego może być przeprowadzony na dwa podstawowe sposoby. W wariacie z **decymacją w czasie** (DIT, ang. *Decimation in Time*) dzieli się próbki sygnału wejściowego na te o indeksach parzystych i nieparzystych, wykonuje DFT osobno na każdym zbiorze, a następnie odtwarza widmo całego sygnału z widm cząstkowych [6]. W wariacie z **decymacją w częstotliwości** (DIF, ang. *Decimation in Frequency*) postępuje się odwrotnie — dzieli się nie próbki wejściowe, lecz prążki widma wyjściowego [6].

Konsekwencją podziału jest to, że dane trafiają do obliczeń w innej kolejności niż ta pierwotna w sygnale wejściowym, więc na którymś etapie algorytmu należy je uporządkować. W algorytmach DIT robi się to na wejściu, przed właściwymi obliczeniami, i wynik otrzymuje się w kolej-

ności naturalnej [9]. W DIF jest odwrotnie — porządkowany jest dopiero wynik, na co zwrócono uwagę już w oryginalnej pracy [8].

Samo przestawianie można wykonać na kilka różnych sposobów, z czego podstawowy polega na wielokrotnym dzieleniu próbek na parzyste i nieparzyste. Jest to jednak rozwiązanie nieefektywne, ponieważ dane sortowane są wielokrotnie [6]. Okazuje się, że docelową pozycję każdej próbki da się wyznaczyć z użyciem operacji **permutacji bitowo-odwróconej** (ang. *bit-reversal permutation*) [7]. Metoda ta ustawia w odwrotnej kolejności bity indeksu próbki wektora wejściowego [6], co pozwala dość niskim kosztem obliczeniowym poukładać wektor wejściowy w kolejności wymaganej przez algorytm DIT. Warto dodać, że wiele procesorów sygnałowych udostępnia sprzętowy mechanizm adresowania z odwróceniem bitów, co dodatkowo przyspiesza ten etap [6].

Wynik tej operacji dla sygnału o długości $N = 16$ zestawiono w tab. 3.1 [6]. Warto zauważyć, że w celu ponownego ustawienia próbek w pierwotnej kolejności wystarczy raz jeszcze wykonać tę samą operację. W prezentowanej tabeli jako n oznaczono indeks początkowy, a jako n' indeks po operacji bit-reversal.

Tabela 3.1: Permutacja bitowo-odwrócona indeksów próbek dla $N = 16$

n	zapis dwójkowy	bity odwrócone	n'	n	zapis dwójkowy	bity odwrócone	n'
0	0000	0000	0	8	1000	0001	1
1	0001	1000	8	9	1001	1001	9
2	0010	0100	4	10	1010	0101	5
3	0011	1100	12	11	1011	1101	13
4	0100	0010	2	12	1100	0011	3
5	0101	1010	10	13	1101	1011	11
6	0110	0110	6	14	1110	0111	7
7	0111	1110	14	15	1111	1111	15

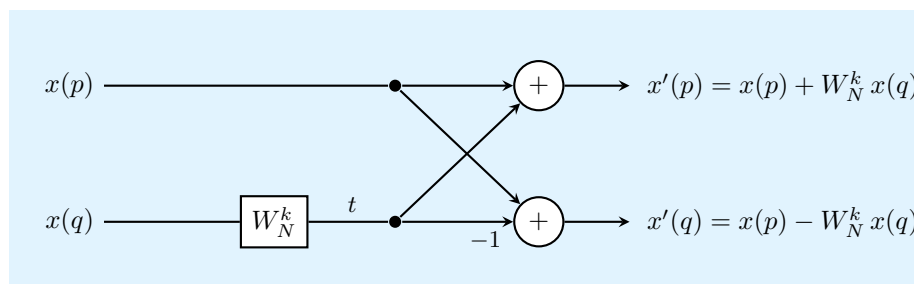
Niezależnie od przyjętego wariantu uporządkowania danych, kolejną elementarną operacją wykonywaną na każdym etapie algorytmu FFT, bazujących na założeniach zaprezentowanych w [8], jest tak zwana **operacja motylkowa** (ang. *butterfly*) [6]. Operacja ta pobiera parę liczb zespolonych $x(p)$ oraz $x(q)$, mnoży drugą z nich przez odpowiedni czynnik obrotu (3.4), a następnie wyznacza sumę i różnicę otrzymanych w ten sposób wartości [7]:

$$x'(p) = x(p) + W_N^k x(q), \quad x'(q) = x(p) - W_N^k x(q) \quad (3.7)$$

gdzie p i q oznaczają pozycje dwóch przetwarzanych elementów w tablicy danych, oddalone od siebie o połowę długości bieżącego etapu, a wykładnik k — numer operacji motylkowej w obrębie tego etapu.

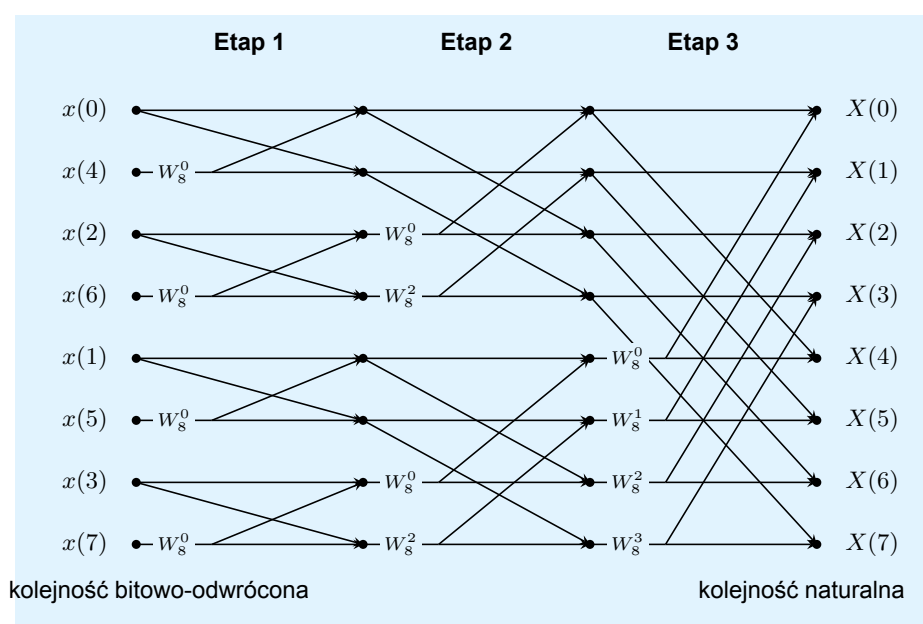
Istotną cechą tak zapisanej operacji jest to, że obie wartości wynikowe mogą zostać zapisane w tych samych komórkach pamięci, z których pobrano argumenty. Dzięki temu cały algorytm można zrealizować bez rezerwowania dodatkowej tablicy na wyniki pośrednie, co pozwala na zaoszczędzenie zasobów sprzętowych [8]. Graficzną reprezentację przepływu sygnałów w operacji motylkowej przedstawiono na rys. 3.1.

Zależności pomiędzy kolejnymi etapami algorytmu wygodnie jest przedstawić w postaci grafu przepływu sygnałów. Na rys. 3.2 zobrazowano pełny przebieg obliczeń dla sygnału o dłu-



Rysunek 3.1: Operacja motylkowa (ang. *butterfly*) algorytmu FFT o podstawie 2, opracowanie własne na podstawie [6]

gości $N = 8$, a więc dla $\log_2 8 = 3$ etapów, z których każdy składa się z $N/2 = 4$ operacji motylkowych.



Rysunek 3.2: Graf przepływu sygnałów algorytmu FFT o podstawie 2 z decymacją w czasie dla $N = 8$; każde skrzyżowanie linii odpowiada operacji motylkowej przedstawionej na rys. 3.1, opracowanie własne na podstawie [6, 9]

Graf ten dobrze ilustruje trzy własności algorytmu, istotne z punktu widzenia jego późniejszej implementacji. Po pierwsze, liczba etapów rośnie logarytmicznie wraz z długością przetwarzanego sygnału, a na każdym z nich wykonywanych jest $N/2$ operacji motylkowych, co daje łącznie $\frac{N}{2} \log_2 N$ mnożeń zespolonych i stanowi bezpośrednie źródło złożoności $O(N \log_2 N)$. Po drugie, próbki wejściowe podawane są w kolejności bitowo-odwróconej, dzięki czemu prążki widma otrzymuje się w kolejności naturalnej. Po trzecie, odległość w pamięci pomiędzy argumentami pojedynczej operacji motylkowej podwaja się z każdym kolejnym etapem — w pierwszym etapie łączone są próbki bezpośrednio sąsiadujące, natomiast w ostatnim oddalone od siebie o $N/2$ pozycji. Ta ostatnia cecha ma bezpośredni wpływ na skuteczność wykorzystania pamięci podręcznej procesora, co szerzej omówiono w Sekcja 3.4.

Operacja bit-reversal jest iteracyjnym sposobem na odpowiednie uporządkowanie tablicy wejściowej przed właściwym przetwarzaniem algorytmu FFT. Sam algorytm można jednak zapisać również z wykorzystaniem rekurencji. Oba warianty, zarówno iteracyjny jak i rekurencyjny są

spotykane praktycznie [9]. Rekurencja dobrze oddaje ideę podziału na coraz mniejsze podproblemy, natomiast w implementacjach nastawionych na wydajność (oraz w tych wykonywanych w systemach o ograniczonej pamięci) częściej stosuje się wariant iteracyjny, w którym wystarczy pętla po kolejnych etapach [9].

Przedstawione powyżej koncepcje stosowane w algorytmie FFT składają się łącznie na kompletny algorytm. Jego minimalną postać, w wariancie radix-2 z decymacją w czasie, przedstawiono na alg. 1 [7].

Algorytm 1 Algorytm FFT o podstawie 2 z decymacją w czasie, na podstawie [6, 7]

Dane wejściowe: wektor próbek x o długości N , gdzie $N = 2^p$

Wynik: wektor x zawierający kolejne prążki widma

```

1: funkcja FFT( $x, N$ )
2:   Operacja BitReversal( $x, N$ )
3:   for  $s = 1, 2, \dots, \log_2 N$  do                                     ▷ kolejne etapy
4:      $m \leftarrow 2^s$                                                  ▷ długość widma składanego w tym etapie
5:     for  $i = 0, m, 2m, \dots, N - m$  do                               ▷ kolejne bloki
6:       for  $j = 0, 1, \dots, m/2 - 1$  do                               ▷ kolejne motylki
7:         wykonaj operację motylkową (3.7)
           na próbkach  $x(i + j)$  oraz  $x(i + j + m/2)$ 
           z czynnikiem obrotu  $W_m^j$ 
8:       end for
9:     end for
10:  end for
11: end funkcja

```

3.2. Opis procesu kompilacji programu komputerowego

3.2.1. Procesory języka programowania

W najnowszych rozwiązaniach dotyczących systemów komputerowych użytkownicy mają szeroki wybór języków programowania, które służą do opisywania idei projektowych człowieka w taki sposób, aby były one możliwe do wykonania przez maszynę. Sprzęt komputerowy nie jest jednak w stanie operować tak wysokopoziomowymi abstrakcjami, jak zmienne, pętle, instrukcje warunkowe itd., które są dostępne w językach programowania. W ich miejsce musi on mieć przygotowane binarne instrukcje maszynowe, które pozbawione są wysokopoziomowej logiki, tak łatwej do zrozumienia dla człowieka. Można zatem powiedzieć, że zanim jakikolwiek program będzie mógł zostać uruchomiony, musi najpierw zostać przetłumaczony na formę, w której będzie mógł być wykonany przez jednostkę centralną komputera. Oprogramowanie systemowe realizujące ten proces translacji określa się ogólnie mianem **procesorów języka** (ang. *language processors*) [10].

Jednym z najpopularniejszych procesorów języka jest **kompilator**, który można w ogólności zdefiniować jako program komputerowy, którego zadaniem jest przetłumaczenie kodu napisanego w jednym języku (języku źródłowym) na równoważny program w innym języku (języku docelowym). Najczęściej sprowadza się to do translacji z wysokopoziomowego języka programowania na język maszynowy, który może być bezpośrednio wykonany przez procesor. Podejście to jest powszechnie stosowane dla języków takich jak **C**, **C++** czy **Java**. Wymagają one uprzedniej kompilacji, ale w zamian oferują programiście ogromną kontrolę nad zarządzaniem pamięcią oraz zasobami sprzętowymi. Dzięki wysokiej wydajności wygenerowanego kodu języki te stanowią standard w wytwarzaniu systemów wbudowanych.

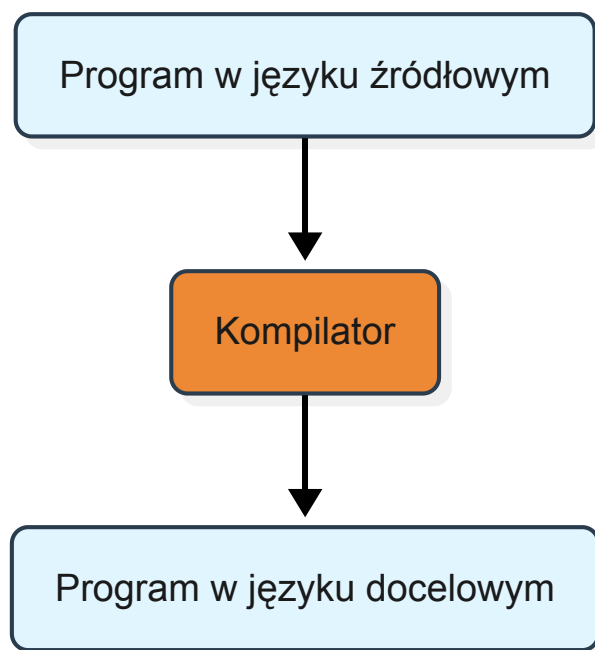
Zupełnie odmiennym mechanizmem charakteryzuje się inny program procesora języka, nazywany **interpreterem**. Rozwiązanie to zamiast generować docelowy plik wykonywalny, analizuje i realizuje instrukcje kodu źródłowego na bieżąco, biorąc pod uwagę aktualnie dostarczone dane wejściowe. Narzut obliczeniowy oraz pamięciowy związany z ciągłą przetwarzaniem instrukcji sprawia, że rozwiązania tego typu rzadko znajdują zastosowanie w systemach wbudowanych. Z tego względu dalsza część niniejszej pracy skupia się wyłącznie na architekturze i mechanizmach działania kompilatorów.

Dalsza analiza teoretyczna w niniejszej pracy zostanie oparta na kompilatorach języka C, ze względu na szerokie zastosowanie tego języka w systemach wbudowanych.

3.2.2. Szczegółowy opis procesu kompilacji

W zastosowaniach niezwiązanych z systemami wbudowanymi wielu użytkowników traktuje kompilator jako wysoką abstrakcję, stosując prosty model „czarnej skrzynki”, co obrazuje rys. 3.3. Zgodnie z tym podejściem narzędzie to zajmuje się wyłącznie tłumaczeniem, a programistę interesuje jedynie to, aby z dostarczonego kodu źródłowego otrzymać działający program docelowy. Przy takim zastosowaniu twórca oprogramowania nie musi poświęcać szczególnej uwagi sposobowi działania kompilatora, traktując go jedynie jako niezbędne narzędzie, pozwalające uruchomić jego program na konkretnym sprzęcie.

W systemach wbudowanych takie uproszczone podejście jest jednak często niewystarczające, ponieważ w takich zastosowaniach powszechnym zjawiskiem są silnie ograniczone zasoby sprzętowe, a nierzadko również ścisły rygor czasowy. Z tego względu programy tworzone przez inżyniera muszą być nie tylko poprawne pod kątem logicznym, ale również niezwykle rozsądnie gospodarować dostępnymi zasobami. Aby świadomie polepszać osiągi i kontrolować ten



Rysunek 3.3: Model "czarnej skrzynki" kompilatora

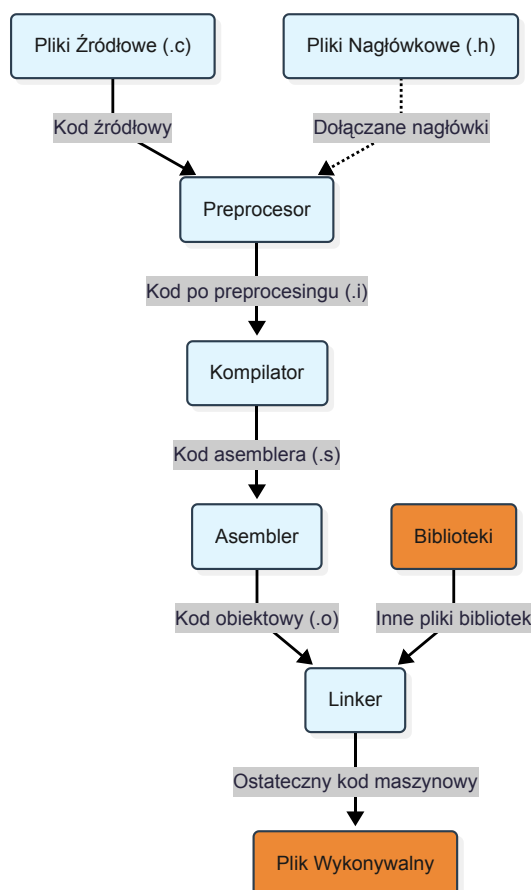
proces, konieczne jest porzucenie modelu czarnej skrzynki i zejście kilka poziomów abstrakcji niżej – począwszy od zrozumienia, jaki zestaw narzędzi jest w ogóle potrzebny do przetworzenia języka.

Na potrzeby niniejszej pracy zostanie przytoczony sposób postępowania opisany w książce [10], przedstawiony na schemacie rys. 3.4. Należy zaznaczyć, że zaprezentowany na rysunku pakiet oprogramowania, powszechnie określany jest przez użytkowników mianem kompilatora, choć w rzeczywistości zawiera w sobie zestaw wielu współpracujących programów komputerowych, w tym ten właściwy kompilator. Jak się okazuje, do utworzenia wykonywalnego programu docelowego wymagane jest najczęściej użycie kilku dodatkowych narzędzi, ułożonych w taki sposób, że wynik przetwarzania jednego jest wejściem dla kolejnego. W klasycznym podejściu proces tworzenia pliku wykonywalnego składa się z następujących faz [11]:

Preprocesor odpowiada za przygotowanie kodu źródłowego przed jego właściwą analizą. Jego początkowym zadaniem jest zebranie programu, który nierzadko bywa podzielony na oddzielne moduły plikowe. Następnie program ten wykonuje odpowiednie dyrektywy oraz rozwija zdefiniowane przez programistę skróty (makra) do postaci pełnoprawnych instrukcji języka źródłowego.

Kompilator przyjmuje zmodyfikowany przez preprocesor kod i dokonuje jego tłumaczenia. Zazwyczaj wynikiem jego pracy nie jest bezpośredni kod maszynowy, lecz program zapisany w języku assemblera, powiązany z docelową architekturą, na której uruchamiany ma zostać przygotowany program. Generowanie na tym etapie kodu assemblerowego wynika z faktu, iż jako język czytelny dla człowieka jest on znacznie łatwiejszy do wyprodukowania oraz ewentualnego debugowania.

Assembler zajmuje się przetworzeniem kodu assemblerowego na postać zrozumiałą dla maszyny. Wynikiem jego działania jest relokowalny kod maszynowy, zapisany w postaci tak zwanych plików obiektowych. Dodatkowo na tym etapie dla każdego z plików generowana jest tablica symboli, opisująca obiekty posiadające powiązania zewnętrzne [11].



Rysunek 3.4: Klasyczny przebieg systemu przetwarzania języka programowania [10]

Linker łączy wygenerowane pliki obiektowe oraz niezbędne biblioteki w jeden spójny plik wykonywalny. Jego kluczowym zadaniem jest rozwiązanie adresów pamięci dla odwołań zewnętrznych pomiędzy poszczególnymi modułami programu, do czego wykorzystuje wspomniane wcześniej tablice symboli. W procesie tym linker dołącza również kod niezbędnych funkcji pochodzących ze standardowych lub zewnętrznych bibliotek [11].

3.2.3. Wewnętrzna struktura kompilatora

Kompilator wewnętrznie dzieli się na dwie części: analizę (front end), która rozkłada program źródłowy na składowe elementy i na ich podstawie buduje reprezentację pośrednią oraz tablicę symboli, oraz syntezę (back end), która na tej podstawie konstruuje program docelowy [10].

Proces ten przebiega jako sekwencja faz, z których każda przekształca jedną reprezentację programu w inną, a tablica symboli jest wykorzystywana przez wszystkie z nich [10]:

- **Analiza leksykalna** - grupuje znaki programu źródłowego w leksemy i zamienia je na tokeny.
- **Analiza składniowa** - na podstawie tokenów buduje drzewo składniowe odzwierciedlające strukturę gramatyczną programu.
- **Analiza semantyczna** - sprawdza zgodność programu z definicją języka, w tym poprawność typów.
- **Generowanie kodu pośredniego** - tworzy niskopoziomową reprezentację programu, np. w postaci kodu trójadresowego.

- **Optymalizacja kodu** - opcjonalnie usprawnia kod pośredni, tak aby wynikowy kod docelowy był lepszy (np. szybszy lub mniejszy).
- **Generowanie kodu** - tłumaczy reprezentację pośrednią na kod maszyny docelowej, dzieląc m.in. rejestry zmiennym.

W praktycznych implementacjach kilka faz bywa łączonych w jeden przebieg (ang. *pass*) [10].

3.3. Przegląd najpopularniejszych kompilatorów języka C

Na rynku dostępnych jest wiele implementacji kompilatorów języka C, zarówno komercyjnych, jak i rozwijanych w modelu otwartoźródłowym. W praktyce jednak w środowisku systemów opartych na jądrze Linux najczęściej spotyka się dwa rozwiązania: **GNU Compiler Collection (GCC)**, powstały w ramach projektu GNU [12], oraz **Clang**, będący częścią projektu **LLVM**. Samo jądro systemu Linux przez długi okres było kompilowane wyłącznie narzędziami GNU, jednak w ostatnim czasie Clang i LLVM stały się dla nich dostępną alternatywą – kompilowane z ich wykorzystaniem jądra systemu Linux wykorzystują między innymi systemy Android czy ChromeOS [13].

Warto zaznaczyć, że również w systemach wbudowanych, niewykorzystujących systemów operacyjnych, narzędzia GNU czy Clang wraz z LLVM są często spotykane. Oferują one wsparcie dla wielu architektur procesorów, pozwalając na uruchomienie kompilatora na sprzęcie o jednej architekturze (nazywanej również terminem **maszyny hosta**) w celu wygenerowania kodu maszynowego przeznaczonego dla zupełnie innej, docelowej architektury (nazywanej **maszyną targetu**). Proces ten określany jest mianem **kompilacji skrośnej** (ang. *cross-compilation*) [10]. Obydwa wspomniane kompilatory posiadają zestaw narzędzi przeznaczonych do różnych architektur spotykanych w systemach wbudowanych, takich jak **ARM, RISC-V, AVR**.

3.3.1. Kompilator GCC

GCC powstał jako część projektu GNU [12] i jest tradycyjnym, domyślnym kompilatorem większości dystrybucji systemu Linux [4]. Jego architektura rozwiązań bazuje na klasycznym modelu kompilatora, który opisuje Sekcja 3.2.3. GCC dzieli proces tłumaczenia na część analizującą (front end) i część generującą kod wynikowy (back end). Część środkowa, odpowiedzialna za optymalizacje, przekształca program w dwóch własnych, uproszczonych reprezentacjach – **GENERIC** oraz **GIMPLE** – na których wykonywane są kolejne przekształcenia mające na celu poprawienie kodu [14].

3.3.2. Kompilator Clang

Clang pełni rolę części analizującej (front end) dla części syntezy obsługiwanej przez **LLVM**. LLVM zapewnia zestaw narzędzi kompilatorowych, zaprojektowanego tak, aby umożliwić analizę i poprawianie kodu na każdym etapie jego translacji i uruchomienia [15]. Podobnie jak GCC, LLVM przekształca program do własnej, uproszczonej reprezentacji pośredniej, jednak w odróżnieniu od GCC reprezentacja ta została zaprojektowana w taki sposób, aby nie musieć zależeć od konkretnego języka programowania [15]. Dzięki takiej budowie LLVM można podzielić na trzy niezależne części: część analizującą (Clang), wspólną dla wszystkich architektur warstwę optymalizacji oraz część generującą kod maszynowy dla konkretnego procesora. Kod projektu LLVM, w tym Clang, jest udostępniany na licencji Apache License [16].

Oba kompilatory, mimo różnic w budowie wewnętrznej, są ze sobą często porównywane w literaturze pod kątem jakości i wydajności generowanego kodu, w tym w zastosowaniach wbudowanych systemów wbudowanych [17]. Dysponują one bowiem wbudowanymi mechanizmami zaawansowanej optymalizacji kodu. Podstawowym narzędziem do konfiguracji optymalizacji programów w języku C są **flagi kompilacji**, czyli ściśle określony zbiór poleceń określających pożądany stopień optymalizacji programu. Polecenia te używane są właśnie przy procesie translacji kodu źródłowego na kod wykonywalny. Podstawowe mechanizmy optymalizacji kodu, wraz z flagami kompilacji zostaną przedstawione w Sekcja 3.4

3.4. Zaawansowane techniki optymalizacji kodu i flagi kompilacji

Optymalizacja kodu w czasie procesu kompilacji jest to dążenie do takiego przekształcenia programu wynikowego, aby osiągał lepsze wyniki w pewnych metrykach, przy zachowaniu poprawności jego założonego działania [10]. Metrykami tymi są na przykład czas wykonania programu, zużycie zasobów pamięci, wielkość kodu programu, czy w przypadku bardzo dużych projektów czas kompilacji [10]. Warunek zachowania poprawności programu jest jednak nadrzędny — przekształcenie, które zmienia wynik obliczeń, nie jest optymalizacją, lecz błędem [10]. Kompilator nie analizuje przy tym programu na tyle głęboko, aby zastąpić zastosowany przez programistę algorytm innym, na przykład o mniejszej złożoności obliczeniowej; stosuje jedynie niskopoziomowe przekształcenia, polegające na przykład na przekształceniach algebraicznych lub na wykorzystaniu dostępnej architektury jednostki centralnej [10].

Termin "optymalizacja" jest w tym kontekście w pewnym stopniu mylący. Program kompilatora nie jest w stanie zagwarantować, że kod wygenerowany przez niego będzie najszybszym możliwym rozwiązaniem danego zadania [10]. W praktyce chodzi o zestaw przekształceń, które w większości przypadków poprawiają założone metryki dla wielu programów, a nie o rozwiązanie optymalne w sensie matematycznym.

Nie istnieje również jedno kryterium tego, co znaczy „lepszy” kod. Skrócenie czasu wykonania odbywa się zwykle kosztem zwiększenia rozmiaru programu, a przekształcenia poprawiające wydajność utrudniają jednocześnie pracę z debuggerem. W systemach wbudowanych kompromis ten nabiera szczególnego znaczenia, ponieważ ograniczona bywa zarówno moc obliczeniowa, jak i dostępna pamięć. Z tego względu kompilatory udostępniają nie jeden, lecz kilka poziomów optymalizacji, ukierunkowanych na różne cele — ich zestawienie przedstawiono w tab. 3.2.

Na przestrzeni lat zmieniły się także same kryteria tego, co w kodzie warto ograniczać. Przykładowo dawniej, gdy procesory sygnałowe wykonywały operacje mnożenia w większej ilości cykli zegara, opłacało się minimalizować ich liczbę nawet kosztem zwiększenia liczby pozostałych operacji, na przykład dodawania, które wymagało mniej cykli. Obecnie jednak procesory sygnałowe potrzebują na każdą z przytoczonych operacji w zasadzie tyle samo czasu, a przesyłanie danych pomiędzy rejestrami a pamięcią trwa, w większości przypadków, tyle samo co operacje arytmetyczne [6]. Pomimo tego, że w dzisiejszych czasach nie ma konieczności na zwracania uwagi na typ wykonywanych operacji, wciąż pozostaje dążenie do zmniejszania ilości ich wykonania.

3.4.1. Przekształcenia optymalizujące

Metody usuwania nadmiarowości

Programowanie w językach wysokiego poziomu ma zarówno wiele aspektów pozytywnych, takich jak używanie wysokopoziomowych abstrakcji, jak i negatywnych, takich jak wykonywanie zbędnych obliczeń. Właśnie z tej przyczyny spora część operacji w programach jest niepotrzebna i nie wynika z braku wiedzy inżyniera oprogramowania, a z narzutu języka. Takie operacje jak odwołania do elementów tablic czy pól struktur rozwijane są bowiem na etapie kompilacji do szeregu operacji arytmetycznych na komórkach pamięci. Operacje te bywają wspólne dla wielu odwołań, a programista nie ma możliwości ich samodzielnego wyeliminowania [10]. W dalszej części prezentowanej sekcji opisano kilka metod na eliminację niepotrzebnych operacji w czasie trwania programu.

Z **wspólnym podwyrażeniem** (ang. *common subexpression*) mamy do czynienia, gdy wynik operacji jest już znany (na przykład ta sama operacja z tymi samymi składnikami została już przeprowadzona), a jej zmienne się nie zmieniły. Kompilator ma możliwość przed wykonaniem programu uznać, że operacja da ten sam wynik co poprzednio i użyć wartości obliczonej za pierwszym razem, zamiast powtarzać obliczenia [10]. Pokrewną techniką jest **propagacja kopii** (ang. *copy propagation*), czyli zastąpienie zmiennej wartością, którą wcześniej jej przypisano [10].

Jeżeli kompilator zdoła ustalić, że wartość danej zmiennej jest stała, czyli nie zmienia się w czasie programu, może wyznaczyć ją już w czasie kompilacji i wstawić w miejsce odwołań do niej. Przekształcenie to nosi nazwę **zwijania stałych** (ang. *constant folding*) [10].

Przekształcenia pętli programów

Pętle, a w szczególności pętle zagnieżdżone, są dość newralgicznym punktem programów komputerowych, ze względu na możliwie długi czas ich wykonywania. Z tego względu opłaca się zmniejszać liczbę instrukcji wykonywanych w ich wnętrzu, nawet za cenę zwiększenia ilości kodu poza pętlą [10]. Na tej zasadzie opiera się **przenoszenie kodu** (ang. *code motion*), polegające na wyniesieniu przed pętlę tych obliczeń, których wynik nie zmienia się pomiędzy kolejnymi iteracjami.

Kolejnym przekształceniem jest **redukcja siły operacji** (ang. *strength reduction*), polegająca na zastąpieniu kosztownych instrukcji ich szybszymi odpowiednikami. Historycznie technikę tę wykorzystywano dość powszechnie do zamiany mnożenia na proste dodawanie wewnątrz pętli [10, 6]. Współczesne procesory posiadają rozbudowane układy mnożące, które są na tyle wydajne, że z tego konkretnego zastosowania się już korzysta. Redukcja siły wciąż pozostaje przydatna w przypadku powolnych operacji, na przykład dzielenia i modulo, które kompilator może zastępować szybkimi operacjami bitowymi.

Kolejną operacją jest **rozwijanie pętli** (ang. *loop unrolling*), polegające na umieszczeniu kilku kolejnych iteracji w jednym, powiększonej instrukcji pętli. Zabieg ten ogranicza narzut związany z obsługą instrukcji pętli, a jednocześnie zwiększa liczbę operacji wykonywanych w pojedynczym przebiegu, co daje kompilatorowi większe możliwości wykrycia operacji nadających się do równoległego wykonania [10]. Odbywa się to jednak kosztem zwiększenia rozmiaru kodu programu wynikowego.

Wywołania procedur i przydział rejestrów

Optymalizacja nie musi zawsze dotyczyć samych obliczeń. Może również polegać na redukcji kosztów ich obsługi. Jedną z technik realizującą takie przekształcenia jest **wciąganie procedur**

(ang. *procedure inlining*), czyli zastąpienie wywołania procedury jej ciałem [10]. Eliminuje ono narzut związany ze skokiem do innej części programu.

Kolejną taką techniką jest odpowiedni **przydział rejestrów** (ang. *register allocation*). Rejestry są najszybszą pamięcią dostępną w procesorze, jednak ich liczba jest ograniczona, przez co nie wszystkie wartości mogą być w nich przechowywane jednocześnie; pozostałe muszą zostać umieszczone w pamięci operacyjnej. Instrukcje operujące na rejestrach wymagają mniej cykli procesora do wykonania, niż te, które odwołują się do pamięci [10]. Zadanie kompilatora polega więc na takim doborze wartości przechowywanych w rejestrach, aby ograniczyć liczbę odwołań do pamięci — problem ten rozwiązuje się między innymi metodą kolorowania grafu [10].

3.4.2. Hierarchia pamięci i pamięć podręczna

Czas wykonania programu zależy nie tylko od liczby wykonanych instrukcji, ale również od czasu dostępu do danych, potrzebnych do ich wykonania. W związku z tym, że pamięć o dużym rozmiarze i bardzo krótkim czasie dostępu jest praktycznie niespotykana (lub bardzo kosztowna), komputery projektuje się z pamięcią w postaci hierarchii — od nielicznych, szybkich rejestrów procesora, przez pamięć podręczną, aż po wolniejszą pamięć operacyjną [10].

Skuteczność opisanej hierarchii wynika z naturalnego sposobu działania komputerów — programy rzadko odwołują się do pamięci w sposób losowy. Zamiast tego mają tendencję do używania tych samych lub sąsiadujących ze sobą danych. Właśnie dlatego sprzęt pobiera informacje do pamięci podręcznej od razu całymi fragmentami, zwanymi liniami [10]. Odczyt danych już obecnych w pamięci podręcznej, nazywany **trafieniem** (ang. *cache hit*), trwa zaledwie kilka cykli procesora, podczas gdy **chybienie** (ang. *cache miss*) wymaga sięgnięcia do wolniejszej pamięci operacyjnej i trwa nawet kilkadziesiąt razy dłużej [10]. Z tego powodu sposób, w jaki algorytm odwołuje się do kolejnych komórek tablicy, wpływa na czas jego wykonania niezależnie od liczby wykonywanych operacji arytmetycznych.

3.4.3. Podstawowe flagi optymalizacyjne

Opisane wcześniej przekształcenia nie są stosowane przez kompilator samoczynnie. Decyzja, które z nich zostaną wykonane, należy do twórcy oprogramowania. Do realizacji tych technik służą odpowiednie flagi kompilacji. Najczęściej wykorzystuje się w tym celu zbiorcze poziomy optymalizacji, oznaczane jako `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, `-Oz`, `-Og` oraz `-Ofast`. W tab. 3.2 zestawiono ogólne założenia stojące za każdym z tych poziomów, w postaci opisanej w dokumentacjach kompilatorów clang i gcc.

Warto podkreślić, że pod każdym z nich kryje się lista pojedynczych flag, odpowiadających konkretnym przekształceniom optymalizującym program wykonywalny. Flagi pozwalają wybrać zarówno najważniejsze przekształcenia opisane w Sekcja 3.4, jak i inne techniki opisane szerzej w literaturze [10].

Dla przykładu poziom `-O1` włącza w kompilatorze GCC kilkadziesiąt takich flag, między innymi `-ftree-dce` odpowiadającą za eliminację martwego kodu, `-ftree-copy-prop` realizującą propagację kopii czy `-fmove-loop-invariants` przenoszącą niezmienniki poza pętlę [18]. Kolejne poziomy dokładają do tej listy następne przekształcenia. Pełne wykazy flag włączanych przez poszczególne poziomy dostępne są w dokumentacjach kompilatorów [18, 19]. Dokładny zestaw przekształceń włączanych na danym poziomie można również sprawdzić, kompilując program z opcją `-Q --help=optimizers` [18]. Poszczególne flagi można również włączać i wyłączać pojedynczo, niezależnie od wybranego poziomu.

Flagi związane z architekturą procesora

Osobną grupę stanowią flagi określające docelową architekturę sprzętową. Mają one szczególne znaczenie przy kompilacji skrośnej, ponieważ kompilator uruchamiany jest wówczas na maszynie o innej architekturze niż docelowa i nie ma jak ustalić możliwości procesora, na którym program będzie wykonywany.

Współczesne procesory udostępniają między innymi możliwość przetwarzania typu **SIMD** (ang. *Single Instruction, Multiple Data*), pozwalające wykonać jedną operację równocześnie na wielu wartościach umieszczonych w pojedynczym, szerokim rejestrze [20]. Rozwiązanie to sprawdza się zwłaszcza tam, gdzie te same, proste obliczenia powtarzane są wielokrotnie na dużych zbiorach danych, a więc również w algorytmach cyfrowego przetwarzania sygnałów [20]. W procesorach ARM moduł sprzętowy realizujący przetwarzanie SIMD nosi nazwę **NEON** i został szerzej omówiony w Sekcja 4.2.1.

Architekturę wskazuje się opcjami `-march` oraz `-mcpu` [20], a do uruchomienia jednostki NEON konieczne jest dodatkowo podanie opcji `-mfpu=neon`, sygnalizującej dostępność tego mechanizmu [20]. Pominięcie opcji `-mcpu` sprawia, że kompilator przyjmuje wbudowany rdzeń domyślny, przez co wygenerowany kod może działać wolniej [20].

Za samo przekształcenie kodu w tę postać, nazywane **wektoryzacją** (ang. *vectorization*), odpowiada w kompilatorze GCC flaga `-ftree-vectorize`, włączana domyślnie na poziomie `-O2` [18]. Bez poprawnie wskazanej architektury docelowej nie zostanie ona jednak wykonana, mimo wybrania wysokiego poziomu optymalizacji.

Flagi wpływające na dokładność obliczeń

Ostatnią grupę stanowią flagi zmieniające sposób wykonywania działań zmiennoprzecinkowych. Najbardziej znaną z nich jest `-ffast-math`, włączana automatycznie przez poziom `-Ofast` i przez żaden inny [18]. Pozwala ona kompilatorowi między innymi na zmianę kolejności działań zmiennoprzecinkowych, co narusza wymagania standardów języków C i C++ i może prowadzić do otrzymania innego wyniku obliczeń [18].

Flagi tego typu mogą przyspieszyć wykonanie programu, jednak ich zastosowanie wymaga świadomej decyzji. W algorytmach, w których kolejność wykonywania działań wpływa na dokładność wyniku, uzyskane wartości mogą znacząco różnić się od poprawnych wartości [18].

Tabela 3.2: Poziomy optymalizacji w kompilatorach GCC i Clang, na podstawie oficjalnych dokumentacji[18, 19]

Flaga	GCC	Clang
-00	Ustawienie domyślne. Skraca czas kompilacji i ułatwia debugowanie, ponieważ zachowanie programu odpowiada bezpośrednio kodowi źródłowemu.	Brak optymalizacji. Najszybsza kompilacja i najlepiej debugowalny kod wynikowy.
-01 (-O)	Kompilator ogranicza rozmiar kodu i czas jego wykonania, unikając przekształceń wymagających dużego nakładu obliczeniowego. Poziom zalecany dla obszernego kodu generowanego maszynowo, jako kompromis między czasem kompilacji a zużyciem pamięci.	Poziom pośredni między -00 a -02.
-02	Włącza niemal wszystkie przekształcenia niewymagające kompromisu między rozmiarem a szybkością kodu. Wydłuża czas kompilacji względem -01.	Umiarkowany poziom optymalizacji, włączający większość dostępnych przekształceń.
-03	Włącza wszystkie przekształcenia z -02 oraz dodatkowe, dotyczące głównie pętli – między innymi zamianę kolejności pętli zagnieżdżonych, ich dzielenie oraz częściowe rozwijanie.	Odpowiednik -02 rozszerzony o przekształcenia wymagające dłuższego czasu kompilacji lub zwiększające rozmiar kodu w zamian za szybsze wykonanie.
-Os	Bazuje na -02, wyłączając przekształcenia zwykle zwiększające rozmiar kodu, m.in. wyrównanie funkcji i pętli. Dodatkowo włącza wciąganie procedur oraz przekształcenia ukierunkowane na redukcję rozmiaru.	Odpowiednik -02 z dodatkowymi przekształceniami zmniejszającymi rozmiar kodu wynikowego.
-Oz	Agresywna redukcja rozmiaru kodu, zbliżona do -Os. Może zwiększyć liczbę wykonywanych instrukcji, jeśli zajmują one mniej bajtów w pamięci.	Odpowiednik -Os, lecz z dalszą redukcją rozmiaru kodu wynikowego.
-Og	Bazuje na -01, wyłączając przekształcenia najbardziej utrudniające debugowanie – między innymi przenoszenie niezmienników poza pętlę oraz eliminację martwych zapisów.	Zbliżony do -01. Część przekształceń jest wyłączona na rzecz lepszej widoczności zmiennych podczas debugowania.
-Ofast	Włącza wszystkie przekształcenia z -03 oraz dodatkowe, mogące naruszać zgodność ze standardem języka (m.in. -ffast-math).	Włącza wszystkie przekształcenia z -03 wraz z dodatkowymi, agresywnymi optymalizacjami naruszającymi zgodność ze standardem języka. Od wersji 19 Clang oznacza tę flagę jako przestarzałą, ze względu na nieprzewidywalny wpływ na zachowanie poprawnego kodu.

4. PROJEKT SYSTEMU TESTOWEGO I METODYKA PRZEPROWADZONYCH BADAŃ

4.1. Zaimplementowane warianty algorytmu FFT

4.2. Opis platformy sprzętowej

4.2.1. Jednostka Neon

4.3. Założenia projektowe i architektura aplikacji

4.3.1. Weryfikacja poprawności obliczeń z biblioteka zewnętrzna

4.4. Użyte biblioteki i narzędzia programistyczne

4.5. Metodyka przeprowadzonych pomiarów

5. ANALIZA WYNIKÓW PRZEPROWADZONYCH BADAN

5.1. Analiza wpływu flag kompilatora na kod sekwencyjny

Tabela 5.1: Czas wykonania pojedynczej transformacji dla $N = 65536$, kompilator GCC 14.2.0. Mediana z 30 powtórzeń, w milisekundach. Pogrubiono najkrótszy czas w wierszu z pominięciem -00.

Wariant	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	39,409	4,972	4,369	4,417	4,515	4,238	4,378	4,384	4,344	4,307	4,475
radix2_v1	29,635	4,405	3,656	3,713	4,426	3,774	4,160	4,247	4,096	4,087	3,716
radix2_v2_soa	21,050	5,098	4,613	4,752	4,957	4,572	4,724	4,727	4,781	3,599	4,731
radix2_v3_neon	17,121	3,020	2,992	2,978	3,428	2,970	3,085	3,012	3,106	3,397	3,128
radix2_par (1 w.)	29,712	4,375	3,706	3,799	4,388	3,832	4,216	4,240	4,152	4,118	3,654
radix2_par (2 w.)	18,944	3,646	2,801	2,868	3,353	2,862	3,028	3,131	3,072	3,073	2,803
radix2_par (4 w.)	12,788	2,898	2,495	2,490	2,862	2,448	2,574	2,697	2,557	2,923	2,572
radix2_par_v2 (1 w.)	33,094	4,909	3,818	3,762	4,911	3,718	4,539	4,836	4,823	4,771	3,769
radix2_par_v2 (2 w.)	17,952	3,359	2,777	2,783	3,409	2,845	3,106	3,204	3,379	3,288	2,828
radix2_par_v2 (4 w.)	10,348	2,559	2,298	2,288	2,599	2,274	2,521	2,557	2,518	2,535	2,392
radix4	21,291	5,555	5,232	5,124	5,317	4,661	5,548	5,282	5,679	5,631	5,090
radix4_naive	28,456	10,249	10,082	10,305	10,408	9,863	9,703	10,334	9,551	9,619	10,441
radix4_par (1 w.)	21,969	5,562	5,441	5,362	5,551	4,884	5,447	5,473	5,658	5,693	5,370
radix4_par (2 w.)	14,405	4,495	4,414	4,364	4,532	3,820	4,408	4,437	4,614	4,604	4,325
radix4_par (4 w.)	9,775	4,400	4,542	4,421	4,600	3,793	4,394	4,402	4,407	4,456	4,347
splitradix	15,729	6,870	6,004	5,997	6,105	6,233	6,369	5,738	6,211	6,128	6,066
stockham	15,907	5,514	5,336	3,853	5,407	3,821	3,735	3,793	3,776	3,740	3,711
fftw	2,321	2,257	2,307	2,402	2,331	2,374	2,348	2,332	2,375	2,303	2,243

5.2. Analiza wpływu optymalizacji ręcznej programu sekwencyjnego

5.3. Analiza skalowalności rozwiązania wielowątkowego

5.4. Zestawienie zbiorcze i dyskusja wyników

Tabela 5.2: Czas wykonania pojedynczej transformaty dla $N = 65536$, kompilator Clang. Mediana z 30 powtórzeń, w milisekundach. Pogrubiono najkrótszy czas w wierszu z pominięciem -00.

Wariant	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	29,665	4,474	4,492	4,333	4,367	4,382	3,868	3,768	3,746	3,756	4,402
radix2_v1	23,368	3,383	3,558	3,566	3,364	3,558	3,551	3,688	3,598	3,633	3,502
radix2_v2_soa	20,045	4,648	3,567	3,444	4,620	3,432	3,319	3,282	3,256	3,144	3,502
radix2_v3_neon	20,919	3,199	3,168	3,362	3,238	3,378	3,415	3,474	3,362	3,476	3,203
radix2_par (1 w.)	23,707	3,373	3,600	3,561	3,337	3,506	3,631	3,588	3,706	3,729	3,440
radix2_par (2 w.)	15,329	2,677	2,950	2,795	2,858	2,787	2,807	2,876	2,833	3,080	2,920
radix2_par (4 w.)	10,274	2,523	2,457	2,453	2,496	2,744	2,458	2,490	2,496	2,491	2,505
radix2_par_v2 (1 w.)	26,459	3,689	4,073	3,952	3,989	3,955	3,840	3,944	3,800	3,751	3,979
radix2_par_v2 (2 w.)	14,606	2,769	3,004	2,840	2,871	2,800	2,870	2,776	2,828	2,812	2,947
radix2_par_v2 (4 w.)	8,694	2,422	2,369	2,176	2,370	2,348	2,334	2,293	2,512	2,387	2,326
radix4	18,693	5,135	5,168	5,078	5,089	5,122	5,143	4,647	4,621	4,587	5,017
radix4_naive	24,117	12,885	12,804	12,610	12,706	12,728	10,263	12,448	10,061	10,064	12,696
radix4_par (1 w.)	19,176	5,088	5,004	4,934	5,034	4,944	5,145	4,569	4,646	4,724	5,054
radix4_par (2 w.)	12,457	4,291	4,226	4,083	4,250	4,193	4,280	3,754	3,833	3,791	4,294
radix4_par (4 w.)	8,913	4,476	4,440	4,275	4,279	4,202	4,525	3,718	3,939	3,910	4,395
splitradix	15,620	5,919	5,839	5,663	5,672	5,733	5,500	5,663	5,678	5,623	5,796
stockham	16,350	5,478	3,683	3,506	4,817	3,460	3,546	3,345	3,556	3,602	3,464
fftw	2,296	2,347	2,300	2,312	2,325	2,343	2,277	2,317	2,322	2,310	2,276

Tabela 5.3: Zestawy flag użyte w badaniu.

Etykieta	Flagi
00 ...0s	-00, -01, -02, -03, -0s
unroll	-03 -funroll-loops
ffast	-03 -ffast-math
mcpu	-03 -mcpu=cortex-a72
ff+mcpu	-03 -ffast-math -mcpu=cortex-a72
+alias	ff+mcpu + --param vect-max-version-for-alias-checks=30
lto	-03 -flto

6. PODSUMOWANIE I WNIOSKI

6.1. *Wnioski koncowe*

6.2. *kierunek dalszych badan*

SPIS RYSUNKÓW

2.1	Warstwowy model systemu komputerowego	10
2.2	Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].	14
3.1	Operacja motylkowa algorytmu FFT o podstawie 2	23
3.2	Graf przepływu sygnałów algorytmu FFT z decymacją w czasie dla $N = 8$	23
3.3	Model "czarnej skrzynki" kompilatora	26
3.4	Klasyczny przebieg systemu przetwarzania języka programowania [10]	27

SPIS TABEL

3.1	Permutacja bitowo-odwrócona indeksów próbek dla $N = 16$	22
3.2	Poziomy optymalizacji w kompilatorach GCC i Clang, na podstawie oficjalnych dokumentacji[18, 19]	33
5.1	Porównanie czasu wykonania pojedynczej transformaty dla $N = 65536$ między kompilatorami GCC 14.2.0 i Clang. Mediana z 30 powtórzeń, w milisekundach. Pogrubiono najkrótszy czas w wierszu z pominięciem -00.	35

BIBLIOGRAFIA

- [1] Andrew S. Tanenbaum and Herbert Bos. *Systemy operacyjne*. Helion, Gliwice, 2024. Przekład: Radosław Meryk.
- [2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. J. Wiley & Sons, Hoboken, N.J., 8th ed. edition, 2008.
- [3] K. C Wang. *Embedded and Real-Time Operating Systems*. Springer Nature, Cham, 1 edition, 2017.
- [4] Robert Love. *Linux kernel development*. Developer's library : essential references for programming professionals Linux kernel development. Addison Wesley, Place of publication not identified, 3rd ed. edition, 2010.
- [5] Jack Dongarra and Francis Sullivan. Guest editors' introduction to the top 10 algorithms. *Computing in Science & Engineering*, 2(1):22–23, 2000.
- [6] Tomasz P. Zieliński. *Cyfrowe przetwarzanie sygnałów*. Wydawnictwa Komunikacji i Łączności, 2007.
- [7] Thomas H. Cormen, Thomas H. Cormen, Charles Eric Leiserson, Ronald L. Rivest, Clifford Stein, and M.I.T. Press Wydawca. *Introduction to algorithms / Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein*. MIT Press, Cambridge, MA, third edition. edition, 2009.
- [8] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.
- [9] William H. Press and Cambridge University Press. *Numerical recipes : the art of scientific computing / William H. Press [et al.]*. Cambridge University Press, Cambridge [etc, 3rd ed. edition, 2007.
- [10] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, Marek Włodarz, and Wydawnictwo Naukowe PWN Wydawca. *Kompilatory : reguły, metody, narzędzia / Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman ; [przekład Marek Włodarz]*. [Wydawnictwo Naukowe PWN SA], Warszawa, wydanie ii. edition, 2019.
- [11] Peter Prinz and Tony Crawford. *Język C w pigułce / Prinz, Peter*. APN Promise, 2nd edition edition, 2016.
- [12] Richard Stallman et al. The gnu project, 1998.
- [13] The Linux Kernel Documentation. Building Linux with Clang/LLVM. <https://docs.kernel.org/kbuild/llvm.html>. Dostęp: 19.08.2026.
- [14] Diego Novillo. Gcc an architectural overview, current status, and future directions. In *Proceedings of the linux symposium*, volume 2, page 185, 2006.
- [15] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.

- [16] LLVM Project. LLVM developer policy. <https://llvm.org/docs/DeveloperPolicy.html>.
Dostęp: 19.08.2026.
- [17] Mehrdad Poorhosseini, Wolfgang Nebel, and Kim Grüttner. A compiler comparison in the risc-v ecosystem. In *2020 International Conference on Omni-layer Intelligent Systems (COINS)*, pages 1–6. IEEE, 2020.
- [18] Free Software Foundation. Optimization levels – GNAT user’s guide for native platforms. <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>.
- [19] The Clang Team. clang – the Clang C, C++, and Objective-C compiler (Command Guide). <https://clang.llvm.org/docs/CommandGuide/clang.html>.
- [20] ARM Limited. *NEON Programmer’s Guide*, version 1.0 edition, 2013. ARM DEN0018A, ID071613.